

Introduction to Software Engineering

- ➔ Software has become an indispensable technology for business science and engineering.
- ➔ Software is embedded in systems of all kinds – transportation, medical , telecommunications, military, industrial , entertainment, office machines- the list is endless.
- ➔ These things could not have been predicted by anybody in the 1950's
- ➔ Also no one could have foreseen that millions of computer programs would have to be corrected, adapted and enhanced as time passed: that the burden of performing these **maintenance** activities would absorb more people and resources, than all the work applied to creation of new software.
- ➔ As softwares importance has grown, the software community has continually attempted to develop technologies that will make it easier, faster and less expensive to build and maintain high quality programs.
- ➔ The framework that encompasses a process, a set of methods and an array of tools is called as Software Engineering

IEEE definition of Software Engineering: Software Engineering can be defined as the systematic approach to the development, operation, maintenance and retirement of software.

IEEE definition of Software:Software can be defined as computer programs, procedures, rules and possibly associated documentation and data pertaining to the operation of a computer based system

- ➔ Software is designed and built by software engineers.
- ➔ Software is used by virtually everyone in society.
- ➔ Software engineers have a moral obligation to build reliable software that does no harm to other people.
- ➔ Software engineers view "software" as being made up of the programs, documents, and data.
- ➔ Software users are only concerned with whether or not software products meet their expectations and make their tasks easier to complete.

The evolving role of software:

- ➔ Software is both a product and a vehicle for delivering a product .
- ➔ **As a Product:**Computer hardware, network of computers, and the software present in them acts as information transformer
- ➔ **As a vehicle:** Software acts as a basis for the control of the computer (OS,Networking Software,compilers ,software tools and environment etc)
- ➔ Over the last 50 years, there have been dramatic improvements in hardware performance, profound changes in computer architectures, vast increase in memory and storage capacity, which have resulted in more sophisticated and complex computer based systems
- ➔ This sophistication & complexity has also resulted in huge problems for software engineers
- ➔ Because of the computers and softwares industrial society has been transformed into information society
- ➔ Legacy software systems which are decades old which are still in operation, if not modified could create serious problems millions of people all over the world .For example: Billions of dollars were spent in correcting the Y2K problem in Legacy Softwares at the turn of 20th century , and the 21st Century arrived without any disasters .

Important Questions for Software Engineers

- ➔ Why does it take so long to get software finished?
- ➔ Why are development costs so high?
- ➔ Why can't we find all errors before we give the software to our customers?
- ➔ Why do we continue to have difficulty in measuring progress as software is being developed?

- These questions and many others have resulted in the adoption of Software Engineering practice.

Text book definition of Software: Software is

- (1) instructions that when executed provide desired features, function and performance
- (2) data structures that enable the programs to adequately manipulate information
- (3) the documents that describe the operation and use of the programs

Software Characteristics

- Software is engineered or developed, not manufactured.
 - Software is a logical thing rather than a physical thing.
 - Comparison of hardware and software
 - In hardware the costs concentrated in manufacture (raw materials, components, labour etc)
 - In software costs are concentrated in engineering (design, coding etc)
 -
 - In hardware design defects can't be modified
 - In Software defects can be modified
- Software does not wear out, but it does deteriorate.

- Failure rate of hardware is represented by **bath tub** curve
- High failure rate in the initial versions
- When the defects are corrected in the later versions , the failure rate stabilizes.
- Failure rates are high near the end of life of the product because of wear and tear (effects of temperature, humidity, vibrations, physical deterioration etc)

- Software is not susceptible to environmental maladies that cause hardware to wear out which should give the idealized for software
- Since software is changed often, the failure rate shoots up and smooths as the defects are fixed and takes the form of actual curve.

- Currently, most software is still custom-built.
 - Hardware system can be assembled from standard components. The reusable components are created so that the engineers can concentrate on designing innovative products.
 - In the software world, component based software development is very rare and is evolving.

Software Application Types or categories

- **System software**
 - Collection of programs written to service other programs
 - Eg. Compilers, editors, file management utilities process determinate information structures
 - Other system software (OS, drivers, networking software etc) process largely indeterminate data
 - characterized by heavy interaction with computer hardware, usage by multiple users, concurrent operation, sophisticated process management etc
- **Application software**
 - solve specific business need.
 - Apart from data processing applications, it is used to control business functions in real-time (point of sale transaction processing, real-time manufacturing process control).
- **Embedded software**
 - resides within a product or system used to control provide control features and functions.
 - Eg: Microwave oven control, fuel control in automobile, dashboard display, braking systems etc
- **Engineering/Scientific software**
 - Formerly number crunching algorithms, now range from astronomy to volcanology, automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing
 - Modern types of this are Computer aided design, system simulation etc.
- **Product-line software**
 - Provides specific capability for use by many different customers
 - Can also focus on a limited marketplace for example inventory control products
 - or address mass consumer markets like word processing, spreadsheet, Computer graphics, multimedia, entertainment etc
- **Web Applications**
 - In the simplest form a set of linked hypertext files and graphics content
 - In the form of e-commerce and B2B applications, they are evolving into sophisticated computing environments which integrate corporate databases and business applications
- **Artificial intelligence software**
 - makes use of nonnumerical algorithms to solve complex problems
 - Eg: Robotics, expert systems, pattern recognition (image and voice) artificial neural networks, theorem proving

New Software Challenges

- Ubiquitous computing
 - Wireless networking making possible distributed computing
 - Creating software to allow machines of all sizes to communicate with each other across vast networks
- Netsourcing
 - Architecting simple and sophisticated applications that benefit targeted end-user markets worldwide
 - WWW is rapidly becoming computing engine as well as a content provider.
- Open Source
 - Distributing source code for computing applications so customers can make local modifications easily

and reliably

- New economy
 - ➔ Building applications that facilitate mass communication and mass product distribution using evolving concepts

Legacy software

- ➔ Many programs still provide a valuable business benefit, even though they are one or even two decades old.
- ➔ Many legacy systems remain supportive to core business functions and are indispensable to the business. Hence they are characterized by longevity and business criticality.

Quality of Legacy Software

- ➔ Most Legacy Software are of poor quality because of inextensible designs, poor or nonexistent documentation, test cases or results never archived, a poorly managed change history and so on
- ➔ If the legacy software meets the needs of its users and runs reliably, it isn't broken and does not need to be fixed.
- ➔ Legacy systems often evolve for one or more of the following reasons
 - ➔ To adapt to the new computing environment or technology
 - ➔ To implement new business functions
 - ➔ To make it interoperable with more modern systems or databases
 - ➔ To make it viable with a network environment

Software Evolution

- Process by which programs change shape, adapt to the marketplace, and inherit characteristics from preexisting programs
- Law of continuing change
 - ➔ Systems must be continually adapted or they become progressively unsatisfactory
- Law of increasing complexity
 - ➔ As system evolves its complexity increases unless work is done to reduce the complexity
- Law of self-regulation
 - ➔ System evolution is self-regulating with its process and product measures following near normal distributions
- Law of conservation of Organizational Stability
 - ➔ Average effective global activity rate for an evolving systems is invariant over the product lifetime
- Law of conservation of familiarity
 - ➔ As system evolves all stakeholders must maintain their mastery of its content and behavior to achieve satisfactory evolution
- Law of continuing growth
 - ➔ Functional content of system must be continually increased to maintain user satisfaction over its lifetime
- Law of declining quality
 - ➔ System quality will appear to decline unless the system is rigorously maintained and adapted to environment changes
- Feedback system law
 - ➔ System evolution processes must be treated as multi-level, multi-loop, multi-agent feedback systems to achieve significant improvement

Software Myths

- Still believed by many managers and practitioners . Myths appear to be reasonable statements of facts, sometimes containing elements of truth
- **Management Myth**
 - **Myth:** *We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know ?*
 - **Reality:** Are the practitioners aware of its existence and is it used effectively ? Does the book reflect modern Software Engineering practice ? Is it complete and adaptable ? In many cases the answer to all of these questions is no.
 -
 - **Myth:** *If we get behind schedule, we can add more programmers and catch up.*
 - **Reality:** Software development is not a mechanical process like manufacturing. Adding people to late project makes it later. As new people are added , the newcomers need to be trained by the people who are already working on the project, which reduces the productive time available for the project.
- - **Myth:** *If I decide to outsource the software to a third party, I can just relax and let that firm build it.*
 - **Reality:** If an organization does not understand how to manage and control software project themselves, it will be more difficult for the organization to finish the project in time if it outsources software projects.
- **Customer myths**
 - **Myth:** *A general statement of objectives is sufficient to begin writing programs- we can fill in the details later*
 - **Reality:** Complete and stable statement of requirements is not always possible, an ambiguous statement of objectives will create more problems. Unambiguous requirements are developed by effective and continuous communication between customer and developer.
 -
 - **Myth:** *Project requirements continually change, but changes can be easily accommodated because software is flexible.*
 - **Reality:** The impact of the change varies with the time at which it is introduced. When requirement changes are requested early , cost impact is very small. If the change requests are made in the middle or later stages of the project, cost impact would be huge because of additional resources and major design and coding modifications.
- **Practitioner's myth:**
 - **Myth:** *Once we write the program and get it to work, our job is done.*
 - **Reality:** Sooner we begin writing code, the longer it will take us to get it done. Industry data indicate that between 60 and 80 percent of all effort spent on software will be spent after it is delivered to the customer for the first time (Modification requests, defects, bugs, and the resulting change to the code will take more time which is called as maintenance phase).
 -
 - **Myth:** *Until I get the program running, I have no way of assessing its quality.*
 - **Reality:** Formal technical review is an effective Software Quality Assurance mechanism which can be applied from the inception of the project. Software review has been found to be more effective than testing for finding certain class of errors.
 -
 - **Myth:** *The only deliverable work product for a successful project is the working program*
 - **Reality:** Working program is only one of the elements of Software. Documentation provides most important part of the Software to the development team as well as the users of the software

- **Myth:** *Software engineering will make us create voluminous and unnecessary documentation and will slow us down*
- **Reality:** Software engineering is not about creating documents. It is about creating quality. Better quality leads to reduced rework.

→ Software Creation

- Almost every software project is precipitated by a business need (e.g., correct a system defect, adapt system to changing environment, extend existing system, create new system)
- Many times an engineering effort will only succeed if the software created for the project succeeds
- The market will only accept a product that has the software embedded when it meets the customer's stated or unstated needs

MCQ's

1

Which question no longer concerns the modern software engineer?

- A) Why does computer hardware cost so much?
- B) Why does software take a long time to finish?
- C) Why does it cost so much to develop a piece of software?
- D) Why can't software errors be removed from products prior to delivery?

2

Today the increased power of the personal computer has brought about an abandonment of the practice of team development of software.

- A) True
- B) False

3

Software is a product and can be manufactured using the same technologies used for other engineering artifacts.

- A) True
- B) False

4

Software deteriorates rather than wears out because

- A) Software suffers from exposure to hostile environments
- B) Defects are more likely to arise after software has been used often
- C) Multiple change requests introduce errors in component interactions
- D) Software spare parts become harder to order

5

Most software continues to be custom built because

- A) Component reuse is common in the software world.
- B) Reusable components are too expensive to use.
- C) Software is easier to build without using someone else's components.

D) Off-the-shelf software components are unavailable in many application domains.

6

The nature of software applications can be characterized by their information

- A) complexity
- B) content
- C) determinacy
- D) both b and c

7

Modern software applications are so complex that it is hard to develop mutually exclusive category names.

- A) True
- B) False

8

The so called "new economy" that gripped commerce and finance during the 1990s died and no longer influences decisions made by businesses and software engineers.

- A) True
- B) False

9

The functionality of most computer systems does not need to be enhanced the lifetime of the system.

- A) True
- B) False

10

Change cannot be easily accommodated in most software systems, unless the system was designed with change in mind.

- A) True
- B) False

11

Most software development projects are initiated to try to meet some business need.

- A) True
- B) False

12

In general software only succeeds if its behavior is consistent with the objectives of its designers.

- A) True
- B) False

1.2 A generic view of processes

What is software process ?

- When building a software product, the series of predictable steps or the roadmap that help us create a timely high quality software products is software process.
- Software process **provides** stability, control and organization to the activity of building the software product.
- Software processes are adapted to meet the needs of software engineers and managers as they undertake the development of a software product.
- A software process provides a framework for managing activities that can very easily get out of control.
- Different types of projects require different software processes.
- The software engineer's work products (programs, documentation, data) are produced as a consequence of the activities defined by the software process.
- The best indicators of how well a software process has worked are the quality, timeliness, and long-term viability of the resulting software product.

Software Engineering – A Layered Technology

- Software engineering is the establishment and sound engineering principles applied to obtain reliable and efficient software in an economical manner.
- Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.
- Software engineering encompasses a process, management techniques, technical methods, and the use of tools.
- Software Engineering is a layered technology.

→ Quality Focus

The bedrock that supports software engineering is a quality focus. Software Engineering must rest on an organizational commitment to quality. Total Quality Management, Six Sigma, and similar philosophies foster continuous process improvement culture.

→ Process

The foundation for software engineering is the process layer. Process defines a framework that must be established for effective delivery of software technology. The software process forms the basis for management control of software projects for producing work products (models, documents, data, reports forms etc), establishing milestones, ensuring quality, and managing change.

→ Methods

Methods provide the technical **how to's** for building software. The tasks in the methods are like communication, requirements analysis, design modeling, program construction, testing and support etc.

➔ Tools

Tools provide automated or semiautomated support for the process and methods. When tools are integrated so that information created by one tool can be used by another, a system for the support of software development called **computer-aided software engineering** (CASE) is established

A process framework

- ➔ A process framework establishes the foundation for a complete software process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity.
- ➔ The process framework additionally includes **umbrella activities** that are applicable across the entire software process.

- ➔ Referring to the figure above, each framework activity is populated by a set of software engineering actions- a collection of related tasks that produce a major software engineering work product.
- ➔ Each action is populated with individual work tasks

A generic Process Framework

- ➔ **Communication**
Customer collaboration and requirement gathering

➔ **Planning**

Establishes engineering work plan, describes technical risks, lists resource requirements, work products produced, and defines work schedule

➔ **Modeling**

Creation of models to help developers and customers understand the requires and software design

➔ **Construction**

Code generation and testing.

➔ **Deployment**

software delivered for customer evaluation and feedback

Example Activity : Modeling

- ➔ This activity is composed of two software engineering actions – analysis and design
- ➔ Analysis encompasses a set of work tasks – requirement gathering, elaboration, negotiation, specification and validation that lead to creation of the analysis model or requirements specification
- ➔ Design encompasses a set of work tasks – data design, architectural design, interface design and component level design - that create a design model or design specification
- ➔ A task set is a collection of software engineering work tasks related to work products, quality assurance points and project milestones.

Example task set for Requirements gathering action:

For a simple project:

1. Make a list of stakeholders for the project
2. Invite all stakeholders to an informal meeting
3. Ask each stakeholder to make a list of features and functions required.
4. Discuss requirements and build a final list.
5. Prioritize requirements.
6. Note areas of uncertainty.

For a complex project:

1. Make a list of stakeholders for the project
2. Interview each stakeholder separately to determine overall wants and needs
3. Build a preliminary list of functions and features based on stakeholder input.
4. Schedule a series of facilitated requirements gathering meetings.
5. Conduct meetings.
6. Produce informal user scenarios as part of each meeting
7. Refine user scenarios based on stakeholder feedback .
8. Build a revised list of stakeholder requirements

9. Use quality function deployment techniques to prioritize requirements
10. Package requirements so that they can be delivered incrementally.
11. Note constraints and restrictions that will be placed on the system.
12. Discuss methods for validating the system

Both of the task sets achieve requirement gathering, but they are quite different in their depth and formality.

Software Engineering Umbrella Activities

- ➔ Software project tracking and control
Allows team to assess progress and take corrective action to maintain schedule)
- ➔ Risk management
Assess risks that may affect project outcomes or quality
- ➔ Software quality assurance
Activities required to maintain software quality
- ➔ Formal technical reviews
Assess engineering work products to uncover and remove errors before they propagate to next activity
- ➔ Measurement
Define and collect process, project, and product measures to assist the software team in delivering software meeting customer needs
- ➔ Software configuration management
Manage effects of change
- ➔ Reusability management
Defines criteria for work product reuse and establish mechanisms to achieve component reuse
- ➔ Work product preparation and production
Activities to create models, documents, logs, forms, lists, etc.

Differences in Process Models are in

- ➔ Overall flow and level of interdependencies among tasks
- ➔ Degree to which work tasks are defined within each framework activity
- ➔ Degree to which work products are identified and required
- ➔ Manner in which quality assurance activities are applied
- ➔ Manner in which project tracking and control activities are applied
- ➔ Overall degree of detail and rigor of process description
- ➔ Degree to which stakeholders are involved in the project
- ➔ Level of autonomy given to project team
- ➔ Degree to which team organization and roles are prescribed

Agile process Models

- ➔ If process models are applied without adaptation, they can increase the level of bureaucracy and create difficulty for developers and customers.
- ➔ Process models that emphasize process agility and follow a set of principles that lead to a more informal approach to

software process have been proposed in recent years. These **agile process models** emphasize maneuverability and adaptability. They are appropriate for many types of projects (E.g. Web applications)

Capability Maturity Model Integration (CMMI)

The Software Engineering Institute (SEI) at Carnegie Mellon University (CMU) has developed a comprehensive process meta-model called as Capability Maturity Model Integration (CMMI).

The CMMI represents process meta-model in two different ways

1) as a continuous model and 2) as a staged model

The Continuous model:

This representation describes a process in two dimensions as in the figure below

Each process area is assessed against specific goals and practices and is rated according to the following capability levels

→ **Level 0: Incomplete**

Process is not performed or does not achieve all goals defined for.

→ **Level 1: Performed**

Work tasks required to produce required work products are being conducted.

→ **Level 2: Managed**

People doing work have access to adequate resources to get job done, stakeholders are actively involved, work tasks and products are monitored, reviewed, and evaluated for conformance to process description

→ **Level 3: Defined**

The process is tailored from the organizations set of standard processes according to the organizations tailoring guidelines .

→ **Level 4: Quantitatively Managed**

Software process area is controlled and improved using measurement and quantitative assessment. Quantitative objectives for quality and process performance are established.

→ **Level 5: Optimizing**

The process area is adapted and optimized using quantitative (statistical) means to meet changing customer needs and continuous process improvement is enabled by quantitative feedback from the process and testing innovative ideas

The CMMI defines each process area in terms of “**specific goals**” and the “**specific practices**” required to achieve these goals. Specific goals establish characteristics that must exist if the activities implied by a process area are to be effective. Specific practices refine a goal into a set of process related activities.

Eg: **Project planning** is one of the eight process areas defined by the CMMI for the “**project management**” category. The specific goals (SG) and specific practices (SP) defined for project planning are

SG 1 Establish estimates

- SP 1.1-1 Estimate the scope of the project
- SP 1.2-1 Establish estimates of work product and task attributes
- SP 1.3-1 Define project life cycle
- SP 1.4-1 Determine estimates of effort and cost

SG 2 Develop a project plan

- SP 2.1-1 Establish the budget and schedule
- SP 2.2-1 Identify project risks
- SP 2.3-1 Plan for data management
- SP 2.4-1 Plan for project resources
- SP 2.5-1 Plan for needed knowledge and skills
- SP 2.6-1 Plan for stakeholder involvement
- SP 2.7-1 Establish the project plan

SG 3 Obtain commitment to the plan

- SP 3.1-1 Review plans that affect the project
- SP 3.2-1 Reconcile work and resource levels
- SP 3.3-1 Obtain plan commitment

- ➔ In addition to the specific goals and practices, the CMMI also defines a set of five generic goals related practices for each process area.
- ➔ Each of the five generic goals corresponds to one of the five capability levels.
- ➔ To achieve a particular capability level for a given process area, the generic goal for that level and the generic practices that correspond to the process area must be achieved

Eg: Generic goals (GG) and practices (GP) for the **project planning** process area are

GG 1 Achieve specific goals

- GP 1.1 Perform basic practices

GG 2 Institutionalize a managed process

- GP 2.1 Establish an organizational policy
- GP 2.2 Plan the process
- GP 2.3 Provide resources
- GP 2.4 Assign responsibility
- GP 2.5 Train people
- GP 2.6 Manage configurations
- GP 2.7 Identify and involve relevant stakeholders
- GP 2.8 Monitor and control the process
- GP 2.9 Objectively evaluate adherence
- GP 2.10 Review status with higher level management

GG 3 Institutionalize a defined process

GP 3.1 Establish a defined process

GP 3.2 Collect improvement information

GG 4 Institutionalize a quantitatively managed process

GP 4.1 Establish quantitative objectives for the process

GP 4.2 Stabilize subprocess performance

GG 5 Institutionalize an optimizing process

GP 5.1 Ensure continuous process improvement

GP 5.2 Correct root causes of problems

Staged representation of CMMI:

- ➔ This representation defines five maturity levels, rather than five capability levels as in continuous representation.
- ➔ To achieve a **maturity level**, the specific goals and practices associated with a **set of process areas** must be achieved.
- ➔ The relationship between maturity levels and process areas is as shown in the table below

Level	Focus	Process Areas
Optimizing	Continuous Process Improvement	Organizational Innovation and Deployment Causal analysis
Quantitatively Managed	Quantitative management	Organizational process performance Quantitative project management
Defined	Process standardization	Requirement Development Technical solution Product integration Verification Validation Organizational process focus Organizational process definition Organizational training Integrated Project management Integrated supplier management Risk management Decision analysis and resolution Organizational Environment for Integration Integrated Teaming
Managed	Basic project management	Requirement Management Project planning Project monitoring and control Supplier agreement Management Measurement and analysis Process and product quality assurance Configuration management
Performed		

Process Patterns

- ➔ Process patterns are Templates or methods for describing important characteristics of software processes
- ➔ Software teams can combine software patterns to construct processes that best meet the needs of specific projects
- ➔ Patterns can be used to describe an entire process (Eg. prototyping), framework activity (e.g., planning), or a task within a framework activity (e.g. Project-estimating) etc.

Example template for a process pattern (by Ambler)

- ➔ Pattern name
The pattern is given a meaningful name (e.g. **customer-communication**)
- ➔ Intent (objective of pattern)
The objective of the pattern is described briefly.
- ➔ Type
Task patterns define software engineering action or work task (e.g. **requirements gathering**)
Stage patterns define framework activity for the process (e.g. **communication**). Stage pattern consists of multiple task patterns.
Phase patterns define sequence of framework activities that occur within process (e.g. **Prototyping or spiral model**)
- ➔ Initial Context
The conditions under which the pattern applies are described.
The following things have to be documented before starting the pattern
 - 1)organizational or team related activities that have already occurred
 - 2)entry state of the process
 - 3)already existing software engineering or project information
- ➔ Problem
The problem to be solved by the pattern is described.
- ➔ Solution
The implementation of the pattern is described.
- ➔ Resulting context
The conditions that will result once the pattern has been successfully implemented is described.
The following things are documented
 - 1)organizational or team activities that have occurred.
 - 2)Exit state of the process
 - 3)Software Engineering of project information that has been developed.
- ➔ Related patterns
A list of all process patterns that are directly related to this one are provided.
- ➔ Known Uses/Examples
The specific instances in which the pattern is applicable are indicated

Once process patterns have been developed , they can be reused for the definition of process variants can be defined by a software team.

An example process pattern

- ➔ **Pattern name.Prototyping**
- ➔ **Intent.**To build a model that can be assessed iteratively by stakeholders in an effort to identify software requirements

- ➔ **Type.** Phase Pattern.
- ➔ **Initial Context.** The following conditions must be met before the initiation of the pattern:
 - 1)Stakeholders have been identified
 - 2)a mode of communication between the stakeholders and the software team has been established;
 - 3)the problem to be solved is identified by stakeholders
 - 4)an initial understanding of the project scope, basic business requirements, and project constraints has been developed.
- ➔ **Problem**
Requirements are hazy or nonexistent, yet there is clear recognition that there is a problem, and the problem must be addressed with a software solution.
- ➔ **Solution**
A description of the prototyping process is presented here.
- ➔ **Resulting context**
A software prototype that identifies basic requirements is approved by stakeholders. 1)The prototype may evolve through a series of increments to become the production software or 2) the prototype may be discarded and the production software built using some other process pattern.
- ➔ **Related patterns**
Customer-communication; iterative design; iterative development, customer assessment; requirement extraction.
- ➔ **Known Uses/Examples**
Prototyping is recommended when requirements are uncertain.

Process Assessment

- ➔ The existence of a software process does not guarantee that the software will be delivered on time and it will meet customers needs. Various process assessment models or techniques are used to assess the state of a software process. Some of the models and standards are given below
- ➔ **Standard CMMI Assessment Method for Process Improvement (SCAMPI)** provides a five step assessment model that incorporates initiating, diagnosing, establishing, acting, and learning. The SCAMPI uses the CMMI as the basis for assessment.
- ➔ **CMM-Based Appraisal for Internal Process Improvement (CBA IPI)** provides a diagnostic technique for assessing the relative maturity of a software organization, using the SEI CMM (previous version of

- ➔ **SPICE (ISO/IEC 15504)** standard defines a set of requirements for software process assessment. The intent of the standard is to assist organizations in developing an objective evaluation of the efficacy of any defined software process.
- ➔ **ISO 9001:2000 for Software**
 - It is a generic standard that applies to any organization that wants to improve overall quality of the products, systems, or services that it provides.
 - The International Standards Organization has developed the standard to define the requirements for a quality management system that will serve to produce higher quality products and thereby improve customer satisfaction
 - The standard has adopted a “**plan-do-check-act**” cycle that is applied to the quality management elements of a software project.
 - Plan establishes the process objectives, activities, and tasks necessary to achieve high quality software and resultant customer satisfaction.
 - Do implements the software process
 - Check monitors and measures the process
 - Act initiates software process improvement activities that continually work to improve the process.

Personal Software Process (PSP)

- ➔ Every software developer uses some process to build computer software. The process may be haphazard , may change on a daily basis, may not be efficient, effective or even successful.
- ➔ The personal software process (PSP) emphasizes personal measurement of both work product that is produced and the resultant quality of the work product.
- ➔ The PSP process model defines five framework activities: planning, high-level design, high-level design review, development and postmortem.
- ➔ **Planning**
Size and resource estimates based on requirements. In addition a defect estimate is made. All metrics are recorded in worksheets.
- ➔ **High-level design**
External specifications developed for components and component level design is created. Prototype are built when uncertainty exists.
- ➔ **High-level design review**
Formal verification methods used to uncover design errors, metrics maintained for important tasks
- ➔ **Development**
Component level design refined, code is generated, reviewed, compiled, and tested, metric maintained for important tasks and work results
- ➔ **Postmortem**
Effectiveness of processes is determined using measures and metrics collected, results of analysis should provide guidance for modifying the process to improve its effectiveness

PSP stresses the need for each software engineer to identify errors early and, as important, to understand the types of errors that he is likely make. This is accomplished by a rigorous assessment activity performed on all work products produced by the software engineer.

Team Software Process

- ➔ The goal of TSP is to build self-directed teams that plan and track their work, establish goals, and own their processes and plans
- ➔ Show managers how to coach and motivate their teams and maintain peak performance
- ➔ Accelerate software process improvement by making CCM Level 5 behavior normal and expected
- ➔ Provide improvement guidance to high-maturity organizations
- ➔ Facilitate university teaching of industrial team skills
- ➔ **TSP defines following framework activities**

- Project launch
- Design
- Implementation
- Integration and system testing
- Postmortem

➔ Scripts

- TSP makes use of variety of scripts, forms and standards that serve to guide team members in their work. **Scripts** define specific process activities and other more detailed work functions that are part of the team process.
- E.g. **Project launch**
- Each project is launched using a sequence of tasks (defined as a script) that enables the team to establish solid basis for starting the project. For project launch the following “**launch script**” recommended.
 - ➔ Review project objectives with management and agree on and document team goals.
 - ➔ Establish team roles
 - ➔ Define the teams development process.
 - ➔ Make a quality plan and set quality targets.
 - ➔ Plan for the needed support facilities
 - ➔ Produce an overall development strategy.
 - ➔ Make development plan for the entire project
 - ➔ Make detailed plan for each engineer for the next phase.
 - ➔ Merge the individual plan into a team plan.
 - ➔ Rebalance team work load to achieve a minimum overall schedule.
 - ➔ Assess project risks and assign tracking responsibility for each key risk.

Process Technology Tools

- ➔ Used to adapt process models to be used by software project team
- ➔ Allow organizations to build automated models of common process framework, task sets, and umbrella activities
- ➔ These automated models can be used to determine workflow and examine alternative process structures
- ➔ Tools can be used to allocate, monitor, and even control all software engineering tasks defined as part of the process model

Product and Process

→ Duality of product and process

- About every ten years give or take five, the software community redefines the problem by shifting its focus from product issues to process issues.
- Eg: We had structured programming languages (product) followed by structured analysis methods (process)
- Then we had data encapsulation (product) followed by the current emphasis on CMM (process) followed by object-oriented methods, followed by agile software development
- An example in the scientific community is dual nature of light.
- People derive as much satisfaction from the creative process as they do from the end-product. Eg: An artist enjoys the brush strokes as much as the framed result.
- A creative software professional should also derive as much satisfaction from the process as the end-product.

MCQ's

1. Which of the items listed below is not one of the software engineering layers?

- a) Process b) Manufacturing c) Methods d) Tools

2. Software engineering umbrella activities are only applied during the initial phases of software development projects.

- a) True b) False

3. Which of these are the 5 generic software engineering framework activities?

- a) communication, planning, modeling, construction, deployment
b) communication, risk management, measurement, production, reviewing
c) analysis, designing, programming, debugging, maintenance
d) analysis, planning, designing, programming, testing

4. Process models are described as agile because they

- a) eliminate the need for cumbersome documentation
b) emphasize maneuverability and adaptability
c) do not waste development time on planning activities
d) make extensive use of prototype creation

5. Which of these terms are level names in the Capability Maturity Model?

- a) Performed b) Repeated c) Reused d) Optimized e) both a and b

6. Software processes can be constructed out of pre-existing software patterns to best meet the needs of a software project.

- a) True b) False

7. Which of these are standards for assessing software processes?

- a) SEI b) SPICE c) ISO 19002 d) ISO 9001 e) both b and d

8. The best software process model is one that has been created by the people who will actually be doing the work.

a) True b)False

9. Which of these is not a characteristic of Personal Software Process?

- a) Emphasizes personal measurement of work product
- b) Practitioner requires careful supervision by the project manager
- c) Individual practitioner is responsible for estimating and scheduling
- d) Practitioner is empowered to control quality of software work products

10. Which of these are objectives of Team Software Process?

- a) Accelerate software process improvement
- b) Allow better time management by highly trained professionals
- c) Build self-directed software teams
- d) Show managers how to reduce costs and sustain quality
- e) both b and c

11. Process technology tools allow software organizations to compress schedules by skipping unimportant activities.

a) True b)False

12. It is generally accepted that one cannot have weak software processes and create high quality end products.

a) True b)False

2.1 Process Models

Contents

Introduction

Prescriptive process models

Waterfall Model

Incremental process models

- incremental model
- RAD model

Evolutionary process models

- Prototyping
- The spiral model
- The concurrent development model

The Unified Process

- History
- Phases of the UP
- UP work products

Introduction

- ➔ Prescriptive process models prescribe a distinct set of activities, actions, tasks, milestones, and work products required to engineer high quality software.
- ➔ Prescriptive software process models are adapted to meet the needs of software engineers and managers for a specific project.
- ➔ Prescriptive software models provide stability, control, and organization to a process that if not managed can easily get out of control.
- ➔ Framework activities for a particular process model may be organized into a process flow that may be linear, incremental, or evolutionary.
- ➔ The software engineer's work products (programs, documentation, data) are produced as a consequence of the activities defined by the software process.
- ➔ The best indicators of how well a software process has worked are the quality, timeliness, and long-term viability of the resulting software product.

Prescriptive Process Models

- ➔ Prescriptive process models define a distinct set of activities, actions, tasks, milestones and work products that are required to engineer high quality software.
- ➔ Originally proposed to bring order to the chaos of software development
- ➔ They brought order to software engineering work and provide reasonable guidance to software teams
- ➔ Yet, they have not provided a definitive answer to the problems of software development in an ever changing computing environment

Software Processes

- ➔ Every software engineering organization needs to describe a set of framework activities for the processes it adopts
- ➔ Each framework activity needs to be populated with software engineering actions
- ➔ Each engineering action needs to be defined in terms of a task set that defines the work and work products needed to meet the development goals
- ➔ The resulting process model should be adapted to accommodate the nature of the specific project, people doing the work, and the work environment
- ➔ Regardless of the process model selected, software engineers will choose a generic process framework that includes these activities: communication, planning, modeling, construction, and deployment
- ➔ Each process model will prescribe a set of process elements (framework activities, software engineering actions, tasks, work products, quality assurance, and change control mechanisms) and a

workflow (the manner in which the process elements are interrelated)

Software Process Models

Waterfall Model

- ➔ This model is also called as Classic Life Cycle model
- ➔ Oldest paradigm for software engineering
- ➔ This model is used when the requirements of a problem are understood, when work flows from communication to deployment in a linear fashion.
- ➔ Requirements are well defined and stable

Disadvantages of the Waterfall model

- ➔ Real projects can't follow sequential flow
- ➔ Difficult for the customer to give all the requirements completely. Waterfall model can't handle the natural uncertainty at the beginning of any project.
- ➔ The customer must have patience: A working version is available only at the last stages of the project. Any serious design defect or requirements error can't be modified
- ➔ Waterfall model creates blocking states : Some project team members have to wait for the previous stage work to be finished

Incremental Models

- ➔ We may have to give only limited set of functionality or modules to the customer quickly and give the remaining functionality in later software releases.
- ➔ Incremental models use elements of the Waterfall model in an iterative fashion
- ➔ First increment is a core product . Core product is used by customer who can evaluate the product and feedback can be obtained from the customer.
- ➔ Useful when staffing is unavailable (or less budget or more risk of failure), or less time is given to deliver the product.
- ➔ Also can be used to manage technical risks : Eg: when new hardware or tool is required which is not yet released. When the new hardware is released, a new increment of the software can be released in the next increment.

Figure: Linear sequences in a staggered fashion

Eg: Word processing in Incremental process model

- ➔ First increment : basic file management, editing
- ➔ Second increment: sophisticated editing, printing, preview etc
- ➔ Third increment: Tables, colour, spelling , grammar etc

Rapid Application and Development (RAD) Model

- ➔ This model has short development life cycle, high speed adaptation of the Waterfall model
- ➔ Makes heavy use of reusable software components
- ➔ The project time is in the range of 60 to 90 days
- ➔ Generic process framework may be used for the RAD model (Communication, Planning, Modeling, Construction and deployment)
- ➔ Communication: Understand the business problem.
- ➔ Planning: Multiple teams work in parallel on different modules. For this to happen requirements should be well understood.
- ➔ Modeling: There are three types of modeling Business , Data and Process
- ➔ Construction: Preexisting software components (reusable) are used for construction . Automatic code generation is done
- ➔ Deployment: Integration , delivery and feedback are done at this stages

Figure: RAD model

Drawbacks of RAD model:

- ➔ For large projects sufficient human resource should be available since number of teams are required to finish the different modules in parallel.
- ➔ Developers and customers if not committed to rapid fire action, the project is bound to fail.
- ➔ If system is not properly modularized, building reusable components will be difficult.
- ➔ High performance cannot be achieved since tuning of interfaces of various reusable components is difficult
- ➔ If technical risks are high (for example use of an unproven latest technology), the project is bound to fail

Evolutionary Process Models

- ➔ Software evolves over time as it gives more features and functionality which also increases complexity .
- ➔ Business and product requirements change
- ➔ Most of the present day softwares exhibit the features like Tight market delivery deadlines, competition, business pressure
- ➔ For such softwares basic requirements are well understood, details or advanced features are not well defined
- ➔ In such cases Evolutionary process models are used .
- ➔ Evolutionary process models are iterative and produce more complete version with each iteration.

Prototyping Model

- ➔ Customer gives general objectives of software, detailed input, processing, output requirements not given or known.
- ➔ Prototyping model is mainly used help the customer and the software engineer to develop or define clear requirements in an iterative fashion.
- ➔ Starts with Communication stage. Customer and Software Engineer meet. This stage defines overall objectives, identifies known requirements.

- ➔ In the next stage, prototyping iteration is planned quickly and a quick design is done in the next stage.
- ➔ Quick design focuses on end user visible outputs like user interface , display format (paper design) etc
- ➔ In the next stage a prototype is constructed.
- ➔ Then prototype is deployed and evaluated by users. Feedback received from the users is used to refine the requirements.
- ➔ Further iterations are carried out until final requirements are defined.
- ➔ The prototype if well designed may be used in the final product or may be discarded .

Problems with prototyping

- ➔ The customer may feel that the prototype itself may be enhanced to make it into the final product, but the purpose of the prototype is to define clear requirements.
- ➔ Developer may compromise to get a working prototype fast like using simplest but slow algorithm . The same algorithm or data structures may be used by the developer in the final product.

Spiral Model (Proposed by Boehm 1988)

- ➔ Spiral model couples iterative nature of prototyping with the controlled and systematic aspects of the waterfall model
- ➔ Spiral model is risk driven process model generator.
- ➔ Two main features:
 - 1)Cyclic approach: Incrementally growing a systems definition and implementation, with decreasing degree of risk.
 - 2)Set of anchor point milestones: to ensure stakeholder commitment for feasible and mutually satisfactory system solution.
- ➔ In spiral model software is developed in a series of evolutionary releases. Early iterations give paper model or prototype.
- ➔ Each circle around the spiral divided into 5 generic process framework activities
Communication,Planning,Modeling,Construction and Deployment

Figure: Spiral model

- ➔ The activities are performed in a clockwise direction, starting at the centre.
- ➔ Risk is considered as each revolution is made
- ➔ Anchor point milestones (Combination of work products and conditioned attained along the path of the spiral)are noted for each evolutionary pass.
- ➔ First circle around the spiral produces the product specification, the next circle may give a prototype, later circles will produce more sophisticated versions of the software.
- ➔ Each pass results in adjustment to the project plan, cost and schedule are adjusted, adjusts planned number of iterations required.
- ➔ Spiral model can be adopted or used throughout the life cycle of the product. That is, once the product is fully developed and delivered, spiral model can be used for enhancement project also (maintenance) and can be used until the product is retired.
- ➔ Spiral model is not a solution for all kinds of software. The model requires enough expertise in risk assessment.

Concurrent Development Model

- ➔ Similar to spiral model often used in development of client/server applications

Specialized Process Models

- ➔ Component-Based Development

spiral model variation in which applications are built from prepackaged software components called classes

- ➔ Formal Methods Model

rigorous mathematical notation used to specify, design, and verify computer-based systems

- ➔ Aspect-Oriented Programming

provides a process for defining, specifying, designing, and constructing software aspects like user interfaces, security, and memory management that impact many parts of the system being developed

The Unified Process

- Use-case driven, architecture centric, iterative, and incremental software process
- It is an attempt to draw on best features of conventional software process models and implements many features of agile software development
- The UP recognizes the importance of customer communication and uses better way of describing the customers view of a system using use-cases

History

- In the 1990s there were many methods put forward by tens of people for object oriented programming and design of software.
- During 1990s three prominent OO methodologies were unified by its creators Booch, Jacobson and Rumbaugh (OOAD, OOT and OOT) with additional features proposed by other experts in the OO field.
- It was named as the Unified Modeling Language (UML)
- The UML became industry standard in 1997 and is maintained by the Object management group (OMG).
- As of 2006, UML 2.0 is the latest version of the standard.
- The Unified Process was created by Rational corporation as an extension to the UML . The commercial version of the UP is called as Rational Unified Process (RUP) now part of IBM corporation which sells Software tools for implementing RUP .
- The generic model of the RUP is called as UP . There are various open versions of the UP (OpenUP, EPF etc)

Phases of the UP

The phases and the corresponding stages of the Generic process framework are as below

- Inception phase : customer communication and planning
- Elaboration phase: communication and modeling
- Construction phase: construction
- Transition phase : customer delivery and feedback
- Production phase: software monitoring and support

Unified Process Work Products

Inception phase

- Vision document
- Initial use-case model
- Initial project glossary
- Initial business case
- Initial risk assessment
- Project plan (phases and iterations)
- Business model
- Prototypes

Elaboration phase

- Use-case model
- Functional and non-functional requirements
- Analysis model
- Software architecture description
- Executable architectural prototype
- Preliminary design model
- Revise risk list
- Project plan (iteration plan, workflow, milestones)
- Preliminary user manual

Construction phase

- ➔ Design model
- ➔ Software components
- ➔ Integrated software increment
- ➔ Test plan
- ➔ Test cases
- ➔ Support documentation (user, installation, increment)

Transition phase

- ➔ Delivered software increment
 - ➔ Beta test reports
 - ➔ User feedback
-

MCQs

1.The linear sequential model of software development is

- a)A reasonable approach when requirements are well defined.
- b) A good approach when a working program is required quickly.
- c)The best approach to use for projects with large development teams.
- d)An old fashioned model that cannot be used in a modern context.

2.The linear sequential model of software development is also known as the

- a)Classical life cycle model b)Fountain model c)Spiral model d)Waterfall model e)both a and d

3.The incremental model of software development is

- a)A reasonable approach when requirements are well defined.
- b)A good approach when a working core product is required quickly.
- c)The best approach to use for projects with large development teams.
- d)A revolutionary model that is not used for commercial products.

4.The rapid application development model is

- a)Another name for component-based development.
- b) A useful approach when a customer cannot define requirements clearly.
- c)A high speed adaptation of the linear sequential model.
- d)All of the above.

5.Evolutionary software process models

- a)Are iterative in nature
- b)Can easily accommodate product requirements changes
- c)Do not generally produce throwaway systems
- d)All of the above

6.The prototyping model of software development is

- a)A reasonable approach when requirements are well defined.
- b) A useful approach when a customer cannot define requirements clearly.
- c)The best approach to use for projects with large development teams.
- d)A risky model that rarely produces a meaningful product.

7.The spiral model of software development

- a) Ends with the delivery of the software product
- b) Is more chaotic than the incremental model
- c)Includes project risks evaluation during each iteration
- d)All of the above

8.The concurrent development model is

- a) Another name for the rapid application development model.
- b)Often used for the development of client/server applications.
- c)Only used for development of parallel or distributed systems.
- d)Used whenever a large number of change requests are anticipated.

9.The component-based development model is

- a)Only appropriate for computer hardware design.
- b)Not able to support the development of reusable components.
- c)Works best when object technologies are available for support.
- d)Not cost effective by known quantifiable software metrics.

10.The formal methods model of software development makes use of mathematical methods to

- a) Define the specification for computer-based systems
- b)Develop defect free computer-based systems
- c)Verify the correctness of computer-based systems
- d)All of the above

11.Which of these is not one of the phase names defined by the Unified Process model for software development?

- a)Inception phase
- b) Elaboration phase
- c)Construction phase
- d)Validation phase

12.In the Unified Process model requirements are determined iteratively and may span more than one phase of the process.

- a)True
 - b)False
-

Essay Type Questions

Unit 2.1

1. Explain the Waterfall process model. What are its disadvantages ?
Or
Explain Classic Life Cycle process model
2. What are incremental process models ? Discuss the advantages and disadvantages of the RAD model.
3. What are evolutionary process models ? Explain prototyping model.
Or
Differentiate between incremental and evolutionary process models.
4. Explain in brief the Spiral process model.
5. Explain in brief the Concurrent development process model.
6. What are the different phases and work products of the Unified process.
Or
Explain in brief the Unified process model

Unit 2.2

7. Write short notes on a)Requirements engineering b) requirement? c) User requirements
d)System requirements
8. What are functional requirements ? Explain with an example.
Or
Why functional requirements tend to be imprecise, incomplete and inconsistent Explain with an example.

9. What are non-functional requirements ? Explain in brief with examples.
or
Explain Product requirements, Organizational requirements, External requirements with example for each.
10. What are system goals and verifiable non-functional requirements ? Explain with an example.
11. What are domain requirements ? What are the problems specifying domain requirements ? Explain with an example.
12. What are the different problems encountered when specifying user requirements. Explain with an example. What guidelines should be followed when writing user requirements ?
13. Should design information be mentioned in requirements ? Discuss in brief.
14. What are the problems in writing requirements in natural language and what are the alternatives
or
Write short notes on a) Structured language specifications b)Form-based specifications c)Sequence diagrams for requirements specification
15. What are different types Interface specifications. Give an example for Printer interface in Java Language.
16. Write short notes on IEEE requirements standard (IEEE/ANSI 830-1998 (IEEE, 1998) .Explain the requirements document structure in brief.

---**---

2.2 Software Requirements

Requirements engineering

- ➔ The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.
- ➔ The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process.

What is a requirement?

- ➔ It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification.
- ➔ May be the basis for a bid for a contract - therefore must be open to interpretation;
- ➔ May be the basis for the contract itself - therefore must be defined in detail;
- ➔ Both these statements may be called requirements.

Types of requirement

- ➔ **User requirements**
Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.
- ➔ **System requirements**
A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

Functional and non-functional requirements

- ➔ **Functional requirements**
Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
- ➔ **Non-functional requirements**
constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
- ➔ **Domain requirements**
Requirements that come from the application domain of the system and that reflect characteristics of that domain.
- ➔ **Functional requirements**
Describe functionality or system services.
Depend on the type of software, expected users and the type of system where the software is used.
Functional user requirements may be high-level statements of what the system should do but functional system requirements should describe the system services in detail.
- ➔ **The LIBSYS system**
A library system that provides a single interface to a number of databases of articles in different libraries. Users can search for, download and print these articles for personal study.
- ➔ **Examples of functional requirements**
 - The user shall be able to search either all of the initial set of databases or select a subset from it.
 - The system shall provide appropriate viewers for the user to read documents in the document store.
 - Every order shall be allocated a unique identifier (ORDER_ID) which the user shall be able to copy to the account's permanent storage area.
- ➔ **Requirements imprecision**
Problems arise when requirements are not precisely stated.
Ambiguous requirements may be interpreted in different ways by developers and users.
Consider the term 'appropriate viewers'
User intention - special purpose viewer for each different document type;
Developer interpretation - Provide a text viewer that shows the contents of the document.

→ Requirements completeness and consistency

In principle, requirements should be both complete and consistent.

Complete

They should include descriptions of all facilities required.

Consistent

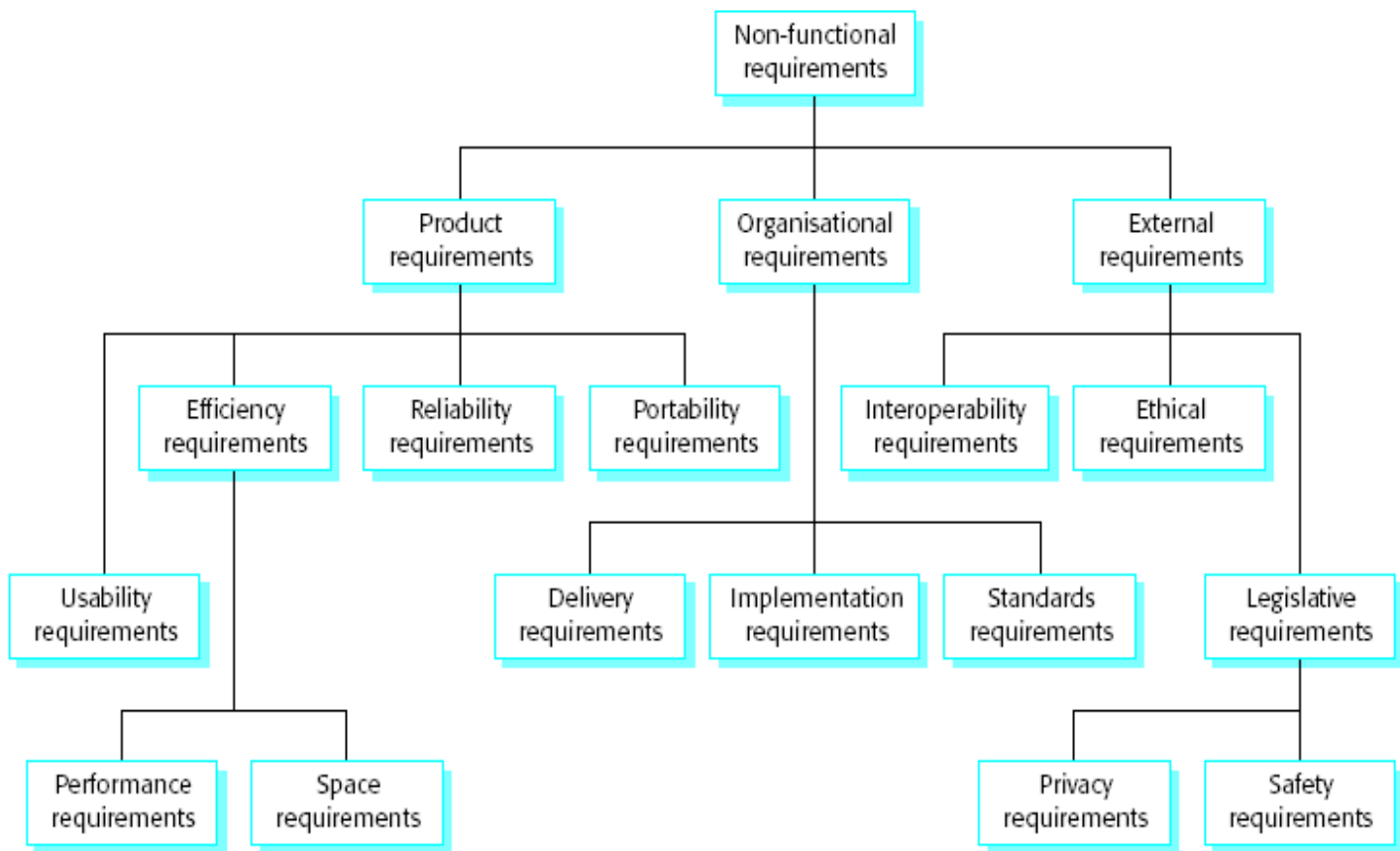
There should be no conflicts or contradictions in the descriptions of the system facilities.

In practice, it is impossible to produce a complete and consistent requirements document.

→ Non-functional requirements

- These define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- Process requirements may also be specified mandating a particular CASE system, programming language or development method.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system is useless.

→ Non-functional classifications



Product requirements

Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.

Organizational requirements

Requirements which are a consequence of organizational policies and procedures e.g. process standards used, implementation requirements, etc.

External requirements

Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Non-functional requirements examples

Product requirement

- The user interface for LIBSYS shall be implemented as simple HTML without frames or Java applets.

Organizational requirement

- The system development process and deliverable documents shall conform to the process and deliverables defined in XYZCo-SP-STAN-95.

External requirement

- The system shall not disclose any personal information about customers apart from their name and reference number to the operators of the system.

Goals and requirements

- Non-functional requirements may be very difficult to state precisely and imprecise requirements may be difficult to verify.

Goal

- A general intention of the user such as ease of use.

Verifiable non-functional requirement

- A statement using some measure that can be objectively tested.
- Goals are helpful to developers as they convey the intentions of the system users.

A system goal example

- The system should be easy to use by experienced controllers and should be organized in such a way that user errors are minimized.

A verifiable non-functional requirement

- Experienced controllers shall be able to use all the system functions after a total of two hours training. After this training, the average number of errors made by experienced users shall not exceed two per day.

→ Domain requirements

- Derived from the application domain and describe system characteristics and features that reflect the domain.
- Domain requirements be new functional requirements, constraints on existing requirements or define specific computations.
- If domain requirements are not satisfied, the system may be unworkable.

→ Library system domain requirements

- There shall be a standard user interface to all databases which shall be based on the Z39.50 standard.
- Because of copyright restrictions, some documents must be deleted immediately on arrival. Depending on the user's requirements, these documents will either be printed locally on the system server for manually forwarding to the user or routed to a network printer.

→ Train protection system

- The deceleration of the train shall be computed as:
 $D_{\text{train}} = D_{\text{control}} + D_{\text{gradient}}$

where D_{gradient} is $9.81\text{ms}^2 * \text{compensated gradient}/\alpha$ and where the values of $9.81\text{ms}^2 / \alpha$ are known for different types of train.

→ Domain requirements problems

→ Understandability

- Requirements are expressed in the language of the application domain;
- This is often not understood by software engineers developing the system.

→ Implicitness

- Domain specialists understand the area so well that they do not think of making the domain requirements explicit.

→ User requirements

- Should describe functional and non-functional requirements in such a way that they are understandable by system users who don't have detailed technical knowledge.
- User requirements are defined using natural language, tables and diagrams as these can be understood by all users.
- **Problems with natural language**
 1. **Lack of clarity**
Precision is difficult without making the document difficult to read.
 2. **Requirements confusion**
Functional and non-functional requirements tend to be mixed-up.
 3. **Requirements amalgamation**
Several different requirements may be expressed together as a single requirement.

→ Eg: LIBSYS requirement

A requirement for a
user accounting
system in LIBSYS

LIBSYS shall provide a financial accounting system that maintains records of all payments made by users of the system. System managers may configure this system so that regular users may receive discounted rates.

A user
requirement for an
editor grid

Grid facilities To assist in the positioning of entities on a diagram, the user may turn on a grid in either centimetres or inches, via an option on the control panel. Initially, the grid is off. The grid may be turned on and off at any time during an editing session and can be toggled between inches and centimetres at any time. A grid option will be provided on the reduce-to-fit view but the number of grid lines shown will be reduced to avoid filling the smaller diagram with grid lines.

→ Requirement problems

User accounting

- Database requirements includes both conceptual and detailed information
- Describes the concept of a financial accounting system that is to be included in LIBSYS;
- However, it also includes the detail that managers can configure this system - this is unnecessary at this level.

Grid Facilities

- Grid requirement mixes three different kinds of requirement
- Conceptual functional requirement (the need for a grid);
- Non-functional requirement (grid units);
- Non-functional UI requirement (grid switching).

Guidelines for writing requirements

1. Invent a standard format and use it for all requirements.
2. Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
3. Use text highlighting to identify key parts of the requirement.
4. Avoid the use of computer jargon.

→ System requirements

More detailed specifications of system functions, services and constraints than user requirements.
They are intended to be a basis for designing the system.
They may be incorporated into the system contract.

Requirements and design

- In principle, requirements should state what the system should do and the design should describe how it does this.
- In practice, requirements and design are inseparable
- A system architecture may be designed to structure the requirements;
- The system may inter-operate with other systems that generate design requirements;
- The use of a specific design may be a domain requirement.

Use of Natural language to write requirements and its problems.

- Ambiguity. The readers and writers of the requirement must interpret the same words in the same way. Natural Language is naturally ambiguous so this is very difficult.
- Over-flexibility. The same thing may be said in a number of different ways in the specification.
- There is no easy way to modularise natural language requirements. It may be difficult to find all related requirements.

→ Alternatives to Natural Language specification

Structured language specifications

The freedom of the requirements writer is limited by a predefined template for requirements.

All requirements are written in a standard way.

The terminology used in the description may be limited.

The advantage is that the most of the expressiveness of natural language is maintained but a degree of uniformity is imposed on the specification.

Form-based specifications

1. Definition of the function or entity.
2. Description of inputs and where they come from.
3. Description of outputs and where they go to.
4. Indication of other entities required.
5. Pre and post conditions (if appropriate).
6. The side effects (if any) of the function.

Graphical models

Graphical models are most useful when you need to show how state changes or where you need to describe a sequence of actions.

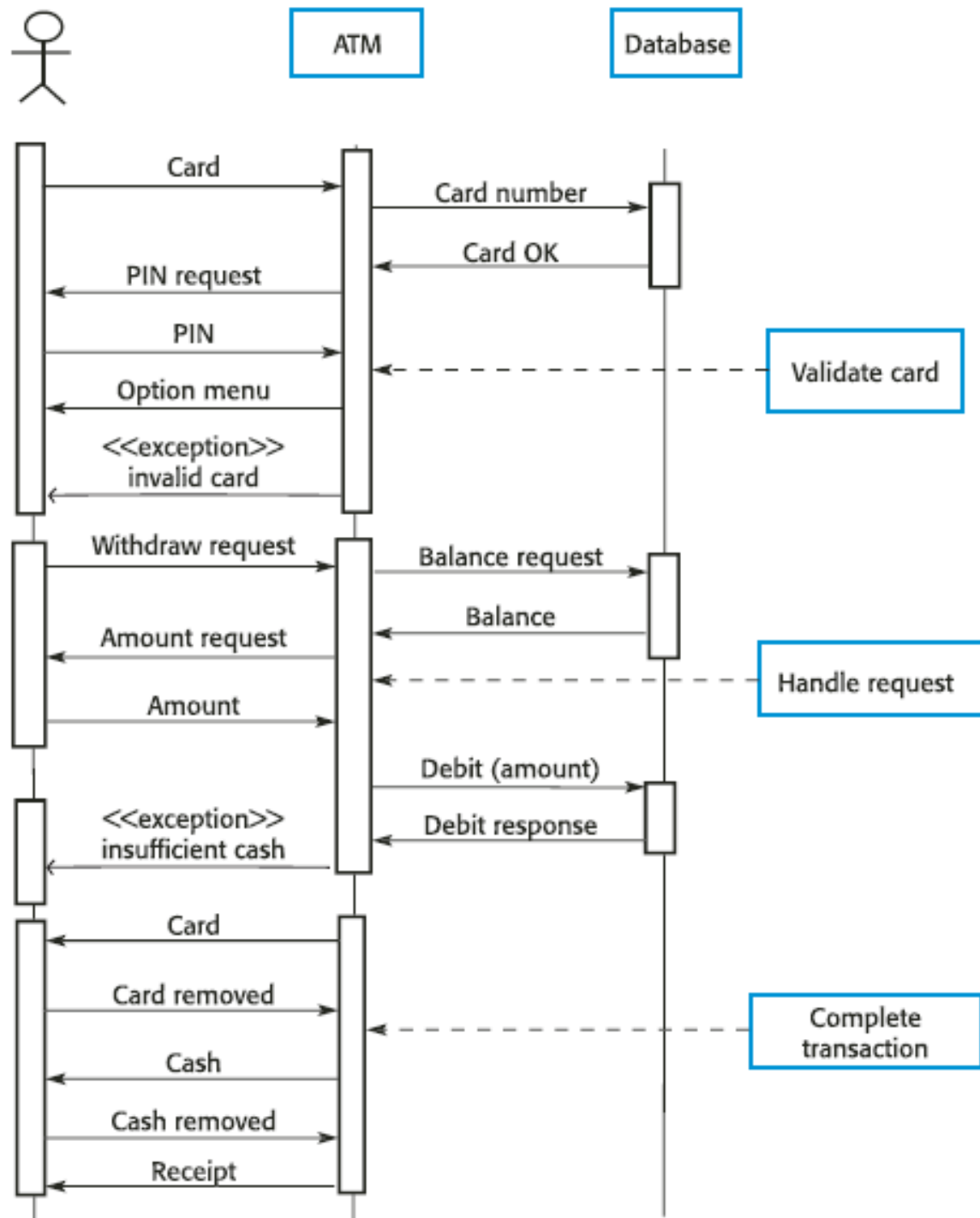
Sequence diagrams

- These show the sequence of events that take place during some user interaction with a system.
- You read them from top to bottom to see the order of the actions that take place.

E.g.: Cash withdrawal from an ATM

1. Validate card: The user's card is validated by checking the card number and user's PIN.
2. Handle request: For a withdrawal, the database must be queried to check the user's balance and to debit the amount withdrawn.
3. Complete transaction: The user's card is returned and, when it is removed, the cash and receipt are delivered.

Sequence diagram of ATM withdrawal



Interface specification

- Most systems must operate with other systems and the operating interfaces must be specified as part of the requirements.
- Three types of interface may have to be defined
 1. Procedural interfaces :These interfaces are sometimes called Application Programming Interfaces (APIs)
 2. Data structures that are exchanged
 3. Data representations(such as the ordering of bits) that have been established for an existing sub-system. These interfaces are most common in embedded, real-time system

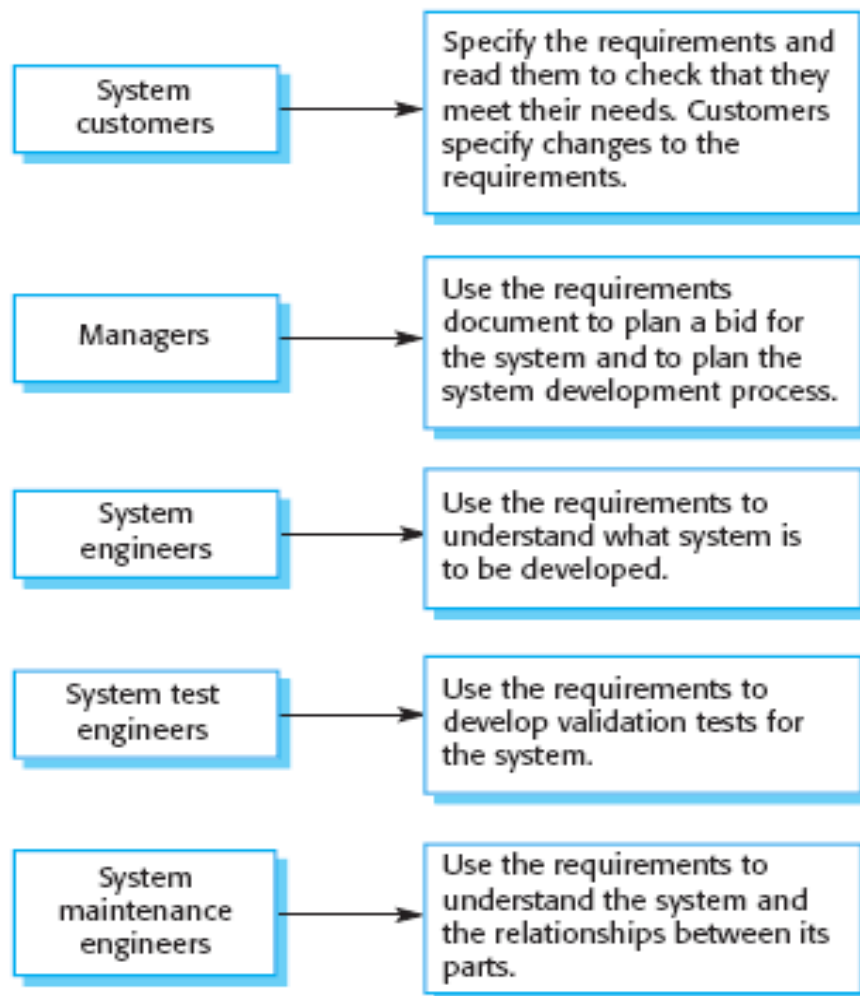
→ PDL interface description

The Java PDL description of a print server interface

```
Interface PrintServer {  
  // defines an abstract printer server  
  // requires: interface Printer, interface PrintDoc  
  // provides: initialize, print, displayPrintQueue, cancelPrintJob, switchPrinter  
  void initialize ( Printer p ) ;  
  void print ( Printer p, PrintDoc d ) ;  
  void displayPrintQueue ( Printer p ) ;  
  void cancelPrintJob (Printer p, PrintDoc d) ;  
  void switchPrinter (Printer p1, Printer p2, PrintDoc d) ;  
} //PrintServer
```

→ The requirements document

- The requirements document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it
- Users of a requirements document



- ➔ **IEEE requirements standard** (IEEE/ANSI 830-1998 (IEEE, 1998))
- ➔ Defines a generic structure for a requirements document that must be instantiated for each specific system.
 - **1. Introduction**
 - 1.1 Purpose of the requirements document
 - 1.2 Scope of the product
 - 1.3 Definitions, acronyms and abbreviations
 - 1.4 References
 - 1.5 Overview of the remainder of the document
 - **2. General description**
 - 2.1 Product perspective
 - 2.2 Product functions
 - 2.3 User characteristics
 - 2.4 General constraints
 - 2.5 Assumptions and dependencies
 - **3. Specific requirements** : cover functional, non-functional and interface requirements
 - **4. Appendices.**
 - **5. Index.**

➔ **Requirements document structure**

- Preface
- Introduction
- Glossary
- User requirements definition
- System architecture
- System requirements specification
- System models
- System evolution
- Appendices
- Index
- Key points
- Requirements set out what the system should do and define constraints on its operation and implementation.
- Functional requirements set out services the system should provide.
- Non-functional requirements constrain the system being developed or the development process.
- User requirements are high-level statements of what the system should do. User requirements should be written using natural language, tables and diagrams.

3.1 Requirements Engineering processes

Contents

Introduction

Feasibility studies

- Requirements elicitation and analysis
- Requirements discovery
- Viewpoints
- Interviewing
- Scenarios
- Use-cases
- Ethnography

Requirements validation

- Requirements reviews

Requirements management

- Enduring and volatile requirements
- Requirements management planning
- Requirements change management

Introduction

- ➔ The processes used for RE vary widely depending on the application domain, the people involved and the organization developing the requirements.
- ➔ However, there are a number of generic activities common to all processes
- ➔ Requirements elicitation;
- ➔ Requirements analysis;
- ➔ Requirements validation;
- ➔ Requirements management.

Spiral model of requirements engineering process

Three-stage activity, activities are organized as iteration around a spiral

Early in the process most effort spent on understanding high-level business and non-functional requirements and user requirements

In the outer rings of the spiral effort will be spent on system requirements and system modeling

→ Feasibility studies

- A feasibility study decides whether or not the proposed system is worthwhile.
- A short focused study that checks
- If the system contributes to organisational objectives;
- If the system can be engineered using current technology and within budget;
- If the system can be integrated with other systems that are used.

→ Feasibility study implementation

- Based on information assessment (what is required), information collection and report writing.
- Questions for people in the organisation
- What if the system wasn't implemented?
- What are current process problems?
- How will the proposed system help?
- What will be the integration problems?
- Is new technology needed? What skills?
- What facilities must be supported by the proposed system?
-

Elicitation and analysis

Sometimes called requirements elicitation or requirements discovery.

Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.

May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called *stakeholders*.

Problems of requirements analysis

Stakeholders don't know what they really want.

Stakeholders express requirements in their own terms.

Different stakeholders may have conflicting requirements.

Organisational and political factors may influence the system requirements.

The requirements change during the analysis process. New stakeholders may emerge and the business environment change.

The requirements spiral

Process activities

Requirements discovery

Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.

Requirements classification and organisation

Groups related requirements and organises them into coherent clusters.

Prioritisation and negotiation

Prioritising requirements and resolving requirements conflicts.

Requirements documentation

Requirements are documented and input into the next round of the spiral.

Requirements discovery

The process of gathering information about the proposed and existing systems and distilling the user and system requirements from this information.

Sources of information include documentation, system stakeholders and the specifications of similar systems.

ATM stakeholders

Bank customers

Representatives of other banks

Bank managers

Counter staff

Database administrators

Security managers

Marketing department

Hardware and software maintenance engineers

Banking regulators

Viewpoints

Viewpoints are a way of structuring the requirements to represent the perspectives of different stakeholders.

Stakeholders may be classified under different viewpoints.

This multi-perspective analysis is important as there is no single correct way to analyse system requirements.

Types of viewpoint

Interactor viewpoints

People or other systems that interact directly with the system. In an ATM, the customer's and the account database are interactor VPs.

Indirect viewpoints

Stakeholders who do not use the system themselves but who influence the requirements. In an ATM, management and security staff are indirect viewpoints.

Domain viewpoints

Domain characteristics and constraints that influence the requirements. In an ATM, an example would be standards for inter-bank communications.

Viewpoint identification

Identify viewpoints using

Providers and receivers of system services;

Systems that interact directly with the system being specified;

Regulations and standards;

Sources of business and non-functional requirements.

Engineers who have to develop and maintain the system;

Marketing and other business viewpoints.

LIBSYS viewpoint hierarchy

Interviewing

In formal or informal interviewing, the RE team puts questions to stakeholders about the system that they use and the system to be developed.

There are two types of interview

Closed interviews where a pre-defined set of questions are answered.

Open interviews where there is no pre-defined agenda and a range of issues are explored with stakeholders.

Interviews in practice

Normally a mix of closed and open-ended interviewing.

Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.

Interviews are not good for understanding domain requirements

Requirements engineers cannot understand specific domain terminology;

Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

Effective interviewers

Interviewers should be open-minded, willing to listen to stakeholders and should not have pre-conceived ideas about the requirements.

They should prompt the interviewee with a question or a proposal and should not simply expect them to respond to a question such as 'what do you want'.

Scenarios

Scenarios are real-life examples of how a system can be used.

They should include

A description of the starting situation;

A description of the normal flow of events;

A description of what can go wrong;

Information about other concurrent activities;

A description of the state when the scenario finishes.

LIBSYS scenario (1)

LIBSYS scenario (2)

Use cases

Use-cases are a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself.

A set of use cases should describe all possible interactions with the system.

Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

Article printing use-case

LIBSYS use cases

Article printing

Print article sequence

Social and organisational factors

Software systems are used in a social and organisational context. This can influence or even dominate the system requirements.

Social and organisational factors are not a single viewpoint but are influences on all viewpoints.

Good analysts must be sensitive to these factors but currently no systematic way to tackle their analysis.

Ethnography

A social scientists spends a considerable time observing and analysing how people actually work.

People do not have to explain or articulate their work.

Social and organisational factors of importance may be observed.

Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

Focused ethnography

Developed in a project studying the air traffic control process

Combines ethnography with prototyping

Prototype development results in unanswered questions which focus the ethnographic analysis.

The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.

Ethnography and prototyping

Scope of ethnography

Requirements that are derived from the way that people actually work rather than the way I which process definitions suggest that they ought to work.

Requirements that are derived from cooperation and awareness of other people's activities.

Requirements validation

Concerned with demonstrating that the requirements define the system that the customer really wants.

Requirements error costs are high so validation is very important

Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking

Validity. Does the system provide the functions which best support the customer's needs?

Consistency. Are there any requirements conflicts?

Completeness. Are all functions required by the customer included?

Realism. Can the requirements be implemented given available budget and technology

Verifiability. Can the requirements be checked?

Requirements validation techniques

Requirements reviews

Systematic manual analysis of the requirements.

Prototyping

Using an executable model of the system to check requirements. Covered in Chapter 17.

Test-case generation

Developing tests for requirements to check testability.

Requirements reviews

Regular reviews should be held while the requirements definition is being formulated.

Both client and contractor staff should be involved in reviews.

Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review checks

Verifiability. Is the requirement realistically testable?

Comprehensibility. Is the requirement properly understood?

Traceability. Is the origin of the requirement clearly stated?

Adaptability. Can the requirement be changed without a large impact on other requirements?

Requirements management

Requirements management is the process of managing changing requirements during the requirements engineering process and system development.

Requirements are inevitably incomplete and inconsistent

New requirements emerge during the process as business needs change and a better understanding of the system is developed;

Different viewpoints have different requirements and these are often contradictory.

Requirements change

The priority of requirements from different viewpoints changes during the development process.

System customers may specify requirements from a business perspective that conflict with end-user requirements.

The business and technical environment of the system changes during its development.

Requirements evolution

Enduring and volatile requirements

Enduring requirements. Stable requirements derived from the core activity of the customer organisation. E.g. a hospital will always have doctors, nurses, etc. May be derived from domain models

Volatile requirements. Requirements which change during development or when the system is in use. In a hospital, requirements derived from health-care policy

Requirements classification

Requirements management planning

During the requirements engineering process, you have to plan:

Requirements identification

How requirements are individually identified;

A change management process

The process followed when analysing a requirements change;

Traceability policies

The amount of information about requirements relationships that is maintained;

CASE tool support

The tool support required to help manage requirements change;

Traceability

Traceability is concerned with the relationships between requirements, their sources and the system design

Source traceability

Links from requirements to stakeholders who proposed these requirements;

Requirements traceability

Links between dependent requirements;

Design traceability

Links from the requirements to the design;

A traceability matrix

CASE tool support

Requirements storage

Requirements should be managed in a secure, managed data store.

Change management

The process of change management is a workflow process whose stages can be defined and information flow between these stages partially automated.

Traceability management

Automated retrieval of the links between requirements.

Requirements change management

Should apply to all proposed changes to the requirements.

Principal stages

Problem analysis. Discuss requirements problem and propose change;

Change analysis and costing. Assess effects of change on other requirements;

Change implementation. Modify requirements document and other documents to reflect change.

Change management

Key points

The requirements engineering process includes a feasibility study, requirements elicitation and analysis, requirements specification and requirements management.

Requirements elicitation and analysis is iterative involving domain understanding, requirements collection, classification, structuring, prioritisation and validation.

Systems have multiple stakeholders with different requirements.

Key points

Social and organisation factors influence system requirements.

Requirements validation is concerned with checks for validity, consistency, completeness, realism and verifiability.

Business changes inevitably lead to changing requirements.

Requirements management includes planning and change management.

MCQs

1.The linear sequential model of software development is

- a)A reasonable approach when requirements are well defined.
- b) A good approach when a working program is required quickly.
- c)The best approach to use for projects with large development teams.
- d)An old fashioned model that cannot be used in a modern context.

2.The linear sequential model of software development is also known as the

- a)Classical life cycle model b)Fountain model c)Spiral model d)Waterfall model e)both a and d

3.The incremental model of software development is

- a)A reasonable approach when requirements are well defined.
- b)A good approach when a working core product is required quickly.
- c)The best approach to use for projects with large development teams.
- d)A revolutionary model that is not used for commercial products.

4.The rapid application development model is

- a)Another name for component-based development.
- b) A useful approach when a customer cannot define requirements clearly.
- c)A high speed adaptation of the linear sequential model.
- d)All of the above.

5.Evolutionary software process models

- a)Are iterative in nature
- b)Can easily accommodate product requirements changes
- c)Do not generally produce throwaway systems
- d)All of the above

6.The prototyping model of software development is

- a)A reasonable approach when requirements are well defined.
- b) A useful approach when a customer cannot define requirements clearly.
- c)The best approach to use for projects with large development teams.
- d)A risky model that rarely produces a meaningful product.

7.The spiral model of software development

- a) Ends with the delivery of the software product
- b) Is more chaotic than the incremental model
- c)Includes project risks evaluation during each iteration
- d)All of the above

8.The concurrent development model is

- a) Another name for the rapid application development model.
- b)Often used for the development of client/server applications.
- c)Only used for development of parallel or distributed systems.
- d)Used whenever a large number of change requests are anticipated.

9.The component-based development model is

- a)Only appropriate for computer hardware design.
- b)Not able to support the development of reusable components.
- c)Works best when object technologies are available for support.
- d)Not cost effective by known quantifiable software metrics.

10.The formal methods model of software development makes use of mathematical methods to

- a) Define the specification for computer-based systems
- b)Develop defect free computer-based systems
- c)Verify the correctness of computer-based systems
- d)All of the above

11.Which of these is not one of the phase names defined by the Unified Process model for software

development?

- a) Inception phase
- b) Elaboration phase
- c) Construction phase
- d) Validation phase

12. In the Unified Process model requirements are determined iteratively and may span more than one phase of the process.

- a) True
 - b) False
-

Essay Type Questions

Unit 2.1

1. Explain the Waterfall process model. What are its disadvantages ?
Or
Explain Classic Life Cycle process model
2. What are incremental process models ? Discuss the advantages and disadvantages of the RAD model.
3. What are evolutionary process models ? Explain prototyping model.
Or
Differentiate between incremental and evolutionary process models.
4. Explain in brief the Spiral process model.
5. Explain in brief the Concurrent development process model.
6. What are the different phases and work products of the Unified process.
Or
Explain in brief the Unified process model

Unit 2.2

7. Write short notes on a) Requirements engineering b) requirement? c) User requirements
d) System requirements
8. What are functional requirements ? Explain with an example.
Or
Why functional requirements tend to be imprecise, incomplete and inconsistent Explain with an example.
9. What are non-functional requirements ? Explain in brief with examples.
or
Explain Product requirements, Organizational requirements, External requirements with example for each.
10. What are system goals and verifiable non-functional requirements ? Explain with an example.
11. What are domain requirements ? What are the problems specifying domain requirements ? Explain with an example.
12. What are the different problems encountered when specifying user requirements. Explain with an example.
What guidelines should be followed when writing user requirements ?

13. Should design information be mentioned in requirements ? Discuss in brief.
14. What are the problems in writing requirements in natural language and what are the alternatives or
Write short notes on a) Structured language specifications b)Form-based specifications c)Sequence diagrams for requirements specification
15. What are different types Interface specifications. Give an example for Printer interface in Java Language.
16. Write short notes on IEEE requirements standard (IEEE/ANSI 830-1998 (IEEE, 1998) .Explain the requirements document structure in brief.

---**---

Unit 3 . System models Notes

Behavioural models

Behavioural models are used to describe the overall behaviour of a system.

Two types of behavioural model are:

Data processing models that show how data is processed as it moves through the system;

State machine models that show the systems response to events.

These models show different perspectives so both of them are required to describe the system's behaviour.

Data-processing models

- Data flow diagrams (DFDs) may be used to model the system's data processing.
- These show the processing steps as data flows through a system.
- DFDs are an intrinsic part of many analysis methods.
- Simple and intuitive notation that customers can understand.
- Show end-to-end processing of data.

Order processing DFD

- DFDs model the system from a functional perspective.
- Tracking and documenting how the data associated with a process is helpful to develop an overall understanding of the system.
- Data flow diagrams may also be used in showing the data exchange between a system and other systems in its environment.

State machine models

- These model the behaviour of the system in response to external and internal events.
- They show the system's responses to stimuli so are often used for modelling real-time systems.
- State machine models show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another.
- Statecharts are an integral part of the UML and are used to represent state machine models.

Statecharts

- Allow the decomposition of a model into sub-models (see following slide).

- A brief description of the actions is included following the ‘do’ in each state.
- Can be complemented by tables describing the states and the stimuli.

Microwave oven model figure

Semantic data models

- Used to describe the logical structure of data processed by the system.
- An entity-relation-attribute model sets out the entities in the system, the relationships between these entities and the entity attributes
- Widely used in database design. Can readily be implemented using relational databases.
- No specific notation provided in the UML but objects and associations can be used.

Library semantic model figure

Data dictionaries

- Data dictionaries are lists of all of the names used in the system models. Descriptions of the entities, relationships and attributes are also included.

Advantages

- Support name management and avoid duplication;
- Store of organisational knowledge linking analysis, design and implementation;
- Many CASE workbenches support data dictionaries.

Data dictionary entries example figure

Object models

- Natural ways of reflecting the real-world entities manipulated by the system
- More abstract entities are more difficult to model using this approach
- Object class identification is recognised as a difficult process requiring a deep understanding of the application domain
- Object classes reflecting domain entities are reusable across systems

Inheritance models

- Organise the domain object classes into a hierarchy.
- Classes at the top of the hierarchy reflect the common features of all classes.
- Object classes inherit their attributes and services from one or more super-classes. these may then be specialised as necessary.
- Class hierarchy design can be a difficult process if duplication in different branches is to be avoided.
- Object models and the UML
- The UML is a standard representation devised by the developers of widely used object-oriented analysis and design methods.
- It has become an effective standard for object-oriented modelling.

Notation

- Object classes are rectangles with the name at the top, attributes in the middle section and operations in the bottom section;
- Relationships between object classes (known as associations) are shown as lines linking objects;
- Inheritance is referred to as generalisation and is shown 'upwards' rather than 'downwards' in a hierarchy.

Library class hierarchy figure

User class hierarchy figure

Multiple inheritance

- Rather than inheriting the attributes and services from a single parent class, a system which supports multiple inheritance allows object classes to inherit from several super-classes.
- This can lead to semantic conflicts where attributes/services with the same name in different super-classes have different semantics.
- Multiple inheritance makes class hierarchy reorganisation more complex.

Multiple inheritance figure

Object aggregation

- An aggregation model shows how classes that are collections are composed of other classes.
- Aggregation models are similar to the part-of relationship in semantic data models.

Object aggregation

Object behaviour modelling

- A behavioural model shows the interactions between objects to produce some particular system behaviour that is specified as a use-case.

Sequence diagrams (or collaboration diagrams) in the UML are used to model interaction between objects.

Sequence diagram for Issue of electronic items figure

4.2 Creating an Architectural Design

Contents

Software Architecture

- What is architectures
- Why is it important

Data Design

- Data Design at the architectural level
- Data Design at the component level

Architectural styles and patterns

- Brief taxonomy of architectural styles
- Architectural patterns
- Organization and refinement

Architectural design

- Representing the system in context
- Defining architypes
- Refining the architecture into components
- Describing the instantiations of the system

Assessing alternative Architectural design

- An alternative Trade-off Analysis method
- Architectural complexity
- Architectural description languages

Introduction

Architectural design represents the structure of the data and program components required to build a computer-based system. A number of architectural "styles" exist. Architectural design begins with data design and proceeds to the derivation of one or more representations of the architectural structure of the system. The resulting architectural model encompasses both the data architecture and the program structure. Alternative architectural patterns are analyzed to determine the structure that is best suited to the customer's requirements. The architectural model is subjected to software quality review like all other design work products.

Software architecture is a representation that enables a software engineer to

Analyze the effectiveness of the design in meeting stated requirements

Consider architectural alternatives

Reduce the risk associated with the construction of the software

Importance of Architecture

Architecture highlights early design decisions that will have a profound impact on the ultimate success of the system as an operational entity

The architecture itself constitutes an intellectually manageable model of how the system is structured and how its components work together

Data Design

The data design translates data objects of the analysis model into data structures at the software component level.

When necessary a database architecture at the application level is created.

Data Design at the Architectural level

Businesses large and small are have normally have dozens of databases serving many applications which may have hundreds of gigabytes of data.

Data Mining

To get useful business-level information from all these databases **data mining** techniques are used, which is also known as knowledge discovery in databases (KDD)

Data mining is difficult to within an existing environment because of multiple databases and different structure of these databases and many other factors like mining tools slowing down the database access for day-to-day operation

An alternative solution called a **data warehouse** adds an additional layer to the data architecture.

Data Warehouse

It is a separate data environment that is not directly integrated with day-to-day applications , but contains all data used by business.

That is, it is a large independent database that has access to the data that are stored in multiple databases of the business.

Data Design at the Component level

- 1.Systematic analysis principles applied to function and behavior should also be applied to data.
- 2.All data structures and the operations to be performed on each should be identified.
- 3.Data dictionary should be established and used to define both data and program design.
- 4.Low level design processes should be deferred until late in the design process.
- 5.Representations of data structure should be known only to those modules that must make direct use of the data contained within in the data structure.
- 6.A library of useful data structures and operations should be developed.
- 7.A software design and its implementation language should support the specification and realization of abstract data types.

Architectural Style Elements

- 1)Set of components (e.g., a database, computational modules)
- 2)Set of connections that enable communication, coordination, and cooperation among components
- 3)Constraints defining how components can be integrated to form the system
- 4)Semantic models that enable designers to understand the overall system properties by analyzing properties of its constituent parts

Comparing Architectural Styles and Patterns

Both impose transformations on the design of an entire system

Scope of architectural pattern is less broad

Patterns impose rules on the architecture describing how the software will deliver some aspect of its functionality at the infrastructure level

Brief Taxonomy of Architectural Styles

Data centered

Data store (e.g., file or database) lies at the center of this architecture and is accessed frequently by other components that modify data

Data flow

Input data is transformed by a series of computational or manipulative components into output data

Call and return

program structure decomposes function into control hierarchy with main program invokes several subprograms

Two substyles exist within this category:

Main program/subprogram architecture

Remote procedure call architecture

Object-oriented

Components of system encapsulate data and operations, communication between components is by message passing

Layered

Several layers are defined, each accomplishing operations that progressively become closer to the machine instruction set

Architectural Patterns

Architectural patterns are different than architectural styles.

For example every house employs a kitchen pattern. The kitchen pattern defines the need for placement of basic kitchen appliances, the need for a sink, the need for storage space and so on.

There is more than a single design for a kitchen. But every design can be conceived within the context of the solution suggested by the kitchen pattern.

Some examples of architectural pattern domains are Concurrency, Persistence, Distribution

Concurrency

- Many applications must handle multiple tasks in a manner that simulates parallelism.
- This is called as concurrency.
- Concurrency can be handled in a number of different ways which can be represented by a different architectural pattern
- E.g.
Operating system process management pattern

A task-scheduler pattern at the application level

Operating system process management pattern

Provides built-in OS features that allow components to execute concurrently

Also manages communication between processes, scheduling and so on to achieve concurrency

A task-scheduler pattern at the application level

Contains a set of active objects that each contains a tick() operation.

The scheduler periodically invokes tick() for each object which then performs the required function

Persistence

Data persists if it survives past the execution of the process that created it.

Persistent data are stored in a database or file .

In object-oriented environments, the values of objects attributes, the general state of the object etc are stored for future retrieval and use.

There are two architectural patterns for persistence in general database management system and application level persistence.

Database management system

Applies the storage and retrieval capability of a DBMS to the application architecture.

Application level persistence

Builds persistence features into the application architecture (e.g. Word processing software that manages its own document structure)

Distribution

The distribution problem addresses the manner in which systems or components within the systems communicate with one another in a distributed environment.

Two elements of distribution are

- 1)the way in which entities connect with one another
- 2)the nature of the communication that occurs

Common pattern for distribution is the **broker pattern**

Broker pattern

acts as a middle-man between the client component and a server component

The client side sends a message to the broker, and the broker completes the connection. Eg. CORBA (Common

object request broker architecture.

Organization and refinement

Design process often leaves a designer with a number of architectural alternatives. The following questions can be used to assess architectural style that has been derived

Control

How is control managed within the architecture?

Does a distinct control hierarchy exist?

How do components transfer control within the system?

How is control shared among components?

What is the control topology?

Is control synchronized or asynchronous?

Data

How are data communicated between components?

Is the flow of data continuous or sporadic?

What is the mode of data transfer?

Do data components exist? If so what is their role?

How do functional components interact with data components?

Are data components active or passive?

How do data and control interact within the system?

Architectural Design

Software to be developed must be put into context (i.e., model external entities and define interfaces)

Identify architectural archetypes (collection of abstractions that must be modeled if the system is to be constructed)

Specify structure of the system by defining and refining the software components needed to implement each archetype

Continue the process iteratively until a complete architectural structure has been derived

Representing the System in Context

Use the architectural context diagram (ACD) to model the manner in which the system interacts with entities external to its boundaries

Systems that interoperate with the target system are represented as

Superordinate systems

Subordinate systems

Peer-level systems

Actors

Superordinate systems

Use the target system as part of some higher level processing scheme

Subordinate systems

Used by the target system to provide data or processing needed to complete the target system functionality

Peer level systems

Produce or consume information needed by peers of the target system

Actors

People or devices that interact with the system to produce or consume information needed for requisite processing

ACD example: SafeHome product

SafeHome product controller and Internet based system are superordinate to the security functional

The surveillance function is a peer systematic

The homeowner and control panels are actors

Sensors are subordinate to the system

Defining Archetypes

May be defined by examining the analysis classes defined in the analysis model

Determine the stable elements of the architecture that will need to be implemented when the system is built

Refining Architecture into Components

Process begins with an examination of the analysis classes for entities from the business domain that must be addressed in the software architecture

Infrastructure components needed to support the business functions

The interfaces depicted in the architecture context diagram may imply specialized components needed to process data that crosses the interfaces

Look for archetypes that are reoccur in several components and create new components that service each repeating design pattern

Architecture Trade-off Analysis Method

Collect scenarios

Elicit requirements, constraints, and environmental description

Describe architectural styles/patterns chosen to address scenarios and requirements (module view, process view, data flow view)

Evaluate quality attributes independently (e.g., reliability, performance, security, maintainability, flexibility, testability, portability, reusability, interoperability)

Identify sensitivity points for architecture (any attributes significantly affected by variation in the architecture)

Critique candidate architectures (from step 3) using the sensitivity analysis (conducted in step 5)

Quantitative Design Analysis Techniques

Spectrum analysis - assesses an architecture using a goodness spectrum from best to worst design

Design selection analysis - architecture is assessed to determine how it compares to an ideal or best case system

Contribution analysis - identifies reasons why one set of design choices gets a lower score than another

Architectural Complexity (similar to coupling)

Sharing dependencies - represent dependence relationships among consumers who use the same resource or producers who produce for the same consumers

Flow dependencies - represent dependence relationships between producers and consumers of resources

Constrained dependencies - represent constraints on the relative flow among a set of components

Architectural Description Language (ADL)

Provides syntax and semantics for describing software architecture

Provides designers with ability to decompose components, combine components, and define interfaces

II. Architectural Design

- Design of context
 - Define the external entities
 - information acquired from analysis model

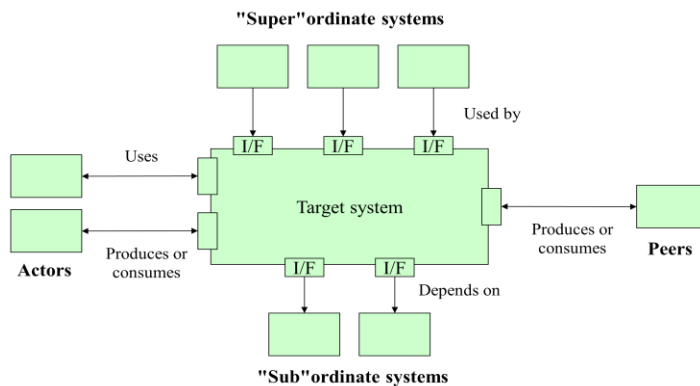
Specifying the structure of system by defining and refining software components

- 1) Represent the system in context
- 2) Define archetypes
- 3) Refine the architecture into components
- 4) Describe instantiations of the system

1. Representing the system in Context(ACD)

->Represents the flow of information into and out of the system, the user interface, and relevant support processing.

->Use ACD to Model the manner in which software interacts with entities external to its boundaries.



Use an architectural context diagram (ACD) that shows

- The identification and flow of all information into and out of a system
- The specification of all interfaces
- Any relevant support processing from/by other systems

An ACD models the manner in which software interacts with entities external to its boundaries

An ACD identifies systems that interoperate with the target system

- Super-ordinate systems
 - Use target system as part of some higher level processing scheme
- Sub-ordinate systems
 - Used by target system and provide necessary data or processing
- Peer-level systems
 - Interact on a peer-to-peer basis with target system to produce or consume data
- Actors
 - People or devices that interact with target system to produce or consume data

2. Defining Archetypes

- Class or Pattern that Represents a core Abstraction.
- Required to design even relatively complex systems.
- Essential in drawing the final or target system
- Only few Archetypes remain enough
 - Archetypes indicate the important abstractions within the problem domain (i.e., they model information)

- An archetype is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system
- It is also an abstraction from a class of programs with a common structure and includes class-specific design strategies and a collection of example program designs and implementations
- Only a relatively small set of archetypes is required in order to design even relatively complex systems
- The target system architecture is composed of these archetypes
 - They represent stable elements of the architecture
 - They may be instantiated in different ways based on the behavior of the system
 - They can be derived from the analysis class model
- The archetypes and their relationships can be illustrated in a UML class diagram

Example-SafeHome security function Diagram

3. Refining Architecture into Components

->Marks the beginning of structure of end system

Initially consider the Classes during Analysis phase

Based on the archetypes, the architectural designer refines the software architecture into components to illustrate the overall structure and architectural style of the system

These components are derived from various sources

- The application domain provides application components, which are the domain classes in the analysis model that represent entities in the real world
- The infrastructure domain provides design components (i.e., design classes) that enable application components but have no business connection
 - Examples: memory management, communication, database, and task management
- The interfaces in the ACD imply one or more specialized components that process the data that flow across the interface

A UML class diagram can represent the classes of the refined architecture and their relationships

- 1.Alarm Processing
- 2.Detector management
- 3.Control Panel Processing
- 4.External Communication management(IFS)

Diagram

4. Describing instantiations of the system

- Arch Design modeled is still at high level.
- We are done with the following aspects

-Context belonging to the system is developed

-Archetype has been defined

-known to the overall system architecture

-Major software components have been identified

- Further refinement is necessary
- To accomplish this an actual instantiation of the architecture is developed.

Diagram

Assessing Architectural Design

1. An Architecture Trade-Off Analysis Method

-Establishes an iterative evaluation process

-Design analysis activities

1. Collect scenarios
2. Elicit Requirements, Constraints and Environment description
3. Describe the architectural styles/patterns
4. Evaluate quality attributes by considering each attribute in isolation
5. Identify the sensitivity of quality attributes to various Architectural attributes for a specific Architectural style
6. Critique candidate architectures using the sensitivity analysis conducted in step 5

2. Architectural Complexity

->Consider dependencies between components for assessing overall complexity

->Dependencies driven by inf/control flow within the system

3 Types:

1. Sharing Dependencies
2. Flow Dependencies
3. Constrained Dependencies

3. Architectural Description Languages

->S/W Architect can draw Arch on UML Notation

->There is a need for a more formal approach to Arch design

->ADL provides a semantic and syntax for describing a s/w arch.

Mapping Data Flow into a S/W Architecture

- 1. Transform Flow
- 2. Transaction Flow
- 3. Transform Mapping
- 4. Transaction Mapping
- 5. Refining the Architectural Design

6.1 Testing Strategies

- A test strategy provides a road map that describes the steps to be conducted as part of testing, when these steps are planned and then undertaken, and how much effort, time, and resources will be required.
- Any testing strategy must incorporate test planning, test case design, test execution, and data collection and evaluation

Importance of a testing strategy:

- Testing often accounts for more project effort than any other software engineering activity.
- If testing is conducted haphazardly, time is wasted and errors may sneak through undetected.
- Hence it is reasonable to establish a systematic strategy for testing software

Steps for testing

- Testing begins "in the small" by focusing on software components and progresses "to the large" by considering system functionality as a whole.
- Initially individual components are tested and debugged.
- After the individual components have been tested and added to the system, integration testing takes place.
- Once the full software product is completed, system testing is performed.

Strategic Approach to Software Testing

Generic characteristics of template for testing:

- To perform effective testing, formal technical reviews should be conducted by the software team. By doing this many errors will be eliminated before testing commences.
- Testing begins at the component level and works outward toward the integration of the entire computer-based system.
- Different testing techniques are appropriate at different points in time.
- The developer of the software conducts testing and may be assisted by independent test groups for large projects.
- Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

Verification and Validation

- Software testing is one element of a broader topic that is often referred to as verification and validation (V&V)
- Verification refers to the set of activities that ensure that software correctly implements a specific function.
- Validation refers to the activities that ensure that the software that has been built is traceable to customer requirements.
- In another way we can state as:
 - Verification - are we building the product right?
 - Validation as - Are we building the right product?
- Verification and Validation activities include wide array of SQA activities like formal technical reviews, quality and configuration audits, performance monitoring, simulation, feasibility study, documentation review, database review, algorithm analysis, development testing, usability testing, qualification testing, installation testing, etc.
- Quality must be built in to the development process, you can't use testing to add quality
- The underlying motivation for testing is to affirm software quality with methods that can be economically and effectively applied to both large-scale and small-scale systems

Organizing for Software Testing

- From a psychological point of view, analysis, design and coding are constructive tasks
- When testing starts, there is an attempt to break the thing that the software engineer has built.
- The builder designs and executes tests that will demonstrate that the program works, rather than uncovering errors.
- There are a number of misconceptions regarding testing, they are
 - 1) The developer should do no testing at all
 - 2) Software is tossed "over the wall" to people to test it mercilessly
 - 3) Testers are not involved with the project until it is time for it to be tested
- The software developer is always responsible for testing the individual components of the program
- In many cases, the developer also conducts integration testing which completes the software architecture.

- Then an independent test group may become involved.
- The role of the Independent Test Group (ITG) is to remove the conflict of interest inherent when the builder is testing his or her own product.
- The developer and ITGC must work together throughout the software project to ensure that thorough tests will be conducted

Software Testing Strategy for Traditional Software Architectures

- Unit Testing - makes heavy use of testing techniques that exercise specific control paths to detect errors in each software component individually
- Integration Testing - focuses on issues associated with verification and program construction as components begin interacting with one another
- Validation Testing - provides assurance that the software validation criteria (established during requirements analysis) meets all functional, behavioral, and performance requirements
- System Testing - verifies that all system elements mesh properly and that overall system function and performance has been achieved

Strategic Testing Issues

1. Specify product requirements in a quantifiable manner before testing starts.
2. Specify testing objectives explicitly.
3. Identify categories of users for the software and develop a profile for each.
4. Develop a test plan that emphasizes rapid cycle testing.
5. Build robust software that is designed to test itself.
6. Use effective formal reviews as a filter prior to testing.
7. Conduct formal technical reviews to assess the test strategy and test cases.
8. Develop a continuous improvement approach for the testing process.

Unit Testing

- Module interfaces are tested for proper information flow.
- Local data are examined to ensure that integrity is maintained.
- Boundary conditions are tested.
- Basis (independent) path are tested.
- All error handling paths should be tested.
- Drivers and/or stubs need to be developed to test incomplete software.

Integration Testing

- Top-down integration testing
- Main control module used as a test driver and stubs are substitutes for components directly subordinate to it.
- Subordinate stubs are replaced one at a time with real components (following the depth-first or breadth-first approach).
- Tests are conducted as each component is integrated.
- On completion of each set of tests and other stub is replaced with a real component.
- Regression testing may be used to ensure that new errors not introduced.
- Bottom-up integration testing
- Low level components are combined into clusters that perform a specific software function.
- A driver (control program) is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure.
- Regression testing - used to check for defects propagated to other modules by changes made to existing program
- Representative sample of existing test cases is used to exercise all software functions.
- Additional test cases focusing software functions likely to be affected by the change.
- Tests cases that focus on the changed software components.
- Smoke testing
- Software components already translated into code are integrated into a build.
- A series of tests designed to expose errors that will keep the build from performing its functions are created.
- The build is integrated with the other builds and the entire product is smoke tested daily (either top-down or bottom integration may be used).

General Software Test Criteria

- Interface integrity - internal and external module interfaces are tested as each module or cluster is added to the

software

- Functional validity - test to uncover functional defects in the software
- Information content - test for errors in local or global data structures
- Performance - verify specified performance bounds are tested

Object-Oriented Unit Testing

- smallest testable unit is the encapsulated class or object
- similar to system testing of conventional software
- do not test operations in isolation from one another
- driven by class operations and state behavior, not algorithmic detail and data flow across module interface

Object-Oriented Integration Testing

- focuses on groups of classes that collaborate or communicate in some manner
- integration of operations one at a time into classes is often meaningless
- thread-based testing - testing all classes required to respond to one system input or event
- use-based testing - begins by testing independent classes (classes that use very few server classes) first and the dependent classes that make use of them
- cluster testing - groups of collaborating classes are tested for interaction errors
- regression testing is important as each thread, cluster, or subsystem is added to the system

Validation Testing

- Pretty much the same for both conventional and object-oriented software
- Focuses on visible user actions and user recognizable outputs from the system
- Validation tests are based on the use-case scenarios, the behavior model, and the event flow diagram created in the analysis model
- Must ensure that each function or performance characteristic conforms to its specification.
- Deviations (deficiencies) must be negotiated with the customer to establish a means for resolving the errors.
- Configuration review or audit is used to ensure that all elements of the software configuration have been properly developed, cataloged, and documented to allow its support during its maintenance phase.

Acceptance Testing

- Making sure the software works correctly for intended user in his or her normal work environment.
- Alpha test - version of the complete software is tested by customer under the supervision of the developer at the developer's site
- Beta test - version of the complete software is tested by customer at his or her own site without the developer being present

System Testing

- Recovery testing - checks the system's ability to recover from failures
- Security testing - verifies that system protection mechanism prevent improper penetration or data alteration
- Stress testing - program is checked to see how well it deals with abnormal resource demands (i.e., quantity, frequency, or volume)
- Performance testing - designed to test the run-time performance of software, especially real-time software

Debugging

- Debugging (removal of a defect) occurs as a consequence of successful testing.
- Some people are better at debugging than others.
- Common approaches:
- **Brute force** - memory dumps and run-time traces are examined for clues to error causes
- **Backtracking** - source code is examined by looking backwards from symptom to potential causes of errors
- **Cause elimination** - uses binary partitioning to reduce the number of locations potential (where errors can exist)

Bug Removal Considerations

- Is the cause of the bug reproduced in another part of the program?
- What "next bug" might be introduced by the fix that is being proposed?
- What could have been done to prevent this bug in the first place?

7.2 Risk Management

- Risks are potential problems that might affect the successful completion of a software project.
- Risks involve uncertainty and potential losses.
- Risk analysis and management is intended to help a software team understand and manage uncertainty during the development process. The important thing is to remember that things can go wrong and to make plans to minimize their impact when they do.
- The work product is called a Risk Mitigation, Monitoring, and Management Plan (RMMM).

Reactive versus Proactive Strategies

Reactive strategies

- Monitor the projects for likely risks.
- Resources are set aside to deal with them should they become actual problems.
- It is also known as fire fighting, project team does nothing until something goes wrong.
- Then, the team flies into action in an attempt to correct the problem rapidly.

Proactive strategies

- Proactive strategy begins long before technical work starts, risks are identified and prioritized by importance
- Then the team builds a plan to avoid risks if they can or minimize them if the risks turn into problems

Software Risks

Two characteristics of software risks are

- Uncertainty – the risk may or may not happen
- Loss – if the risk becomes a reality, unwanted consequences or losses will occur.

Different categories of risks are

Project risks

- Threaten the project plan
- Project risks identify potential budgetary, schedule, personnel, resource, stakeholder, and requirement problems and their impact on a software project

Technical risks

- Threaten product quality and the timeliness of the schedule
- If a technical risk becomes a reality, implementation may become difficult or impossible.
- They identify potential design, implementation, interface, verification and maintenance problems.
- In addition, specification ambiguity, technical uncertainty, technical obsolescence and leading edge technology are also risk factors

Business risks

- Threaten the viability of the software to be built
- Top five business risks are
 - Market risks – building a product that nobody wants
 - Strategic risks – building a product that no longer fits into the overall business strategy for the company.
 - Sales risks – building a product that the sales force doesn't understand how to sell
 - Management risks – losing the support of the senior management
 - Budget risks – losing budgetary or personnel commitment

Another general categorization of risks are

Known risks

- can be uncovered from careful evaluation of current project plan, the business and technical environment etc

Predictable risks

- are extrapolated from past project experience

Unpredictable risks

- They can and do occur, but they are extremely difficult to identify in advance

Risk Identification

- Risk identification is a systematic attempt to specify threats to the project plan.

- By identifying known and predictable risks, the project manager takes a first step toward avoiding them when possible and controlling them when necessary.
- There are two distinct types of risks for each of the categories of risks - Product-specific risks and Generic risks
- Generic risks - are potential threats to every software product
- Product-specific risks - the project plan and software statement of scope are examined to identify any special characteristics of the product that may threaten the project plan

One method of identifying risks is to create a risk item checklist. A sample checklist is as below.

- Product size – risk associated with the overall size of the software to be built or modified
 - Business impact – risks associated with constraints imposed by management or the marketplace
 - Customer characteristics – risks associated with the sophistication of the customer and the developers ability to communicate with the customer in a timely manner
 - Process definition – risks associated with the degree to which the software process has been defined and followed by the development environment
 - Development environment – risks associated with the availability and quality of the tools to be used to build the product
 - Technology to be built - risks associated with the complexity of the system to be built and the newness of the technology that is packaged by the system
 - Staff size and experience - risks associated with the overall technical and project experience of the software of the software engineers who will do the work
- The risk item checklist can be organized as Questions format, where questions relevant to each of the risks can be answered for each software project.
- The answer to these questions allow the planner to estimate the impact of the risk.
- Then a set of risk components and drivers are listed along with the probability of their occurrence
- Finally table of drivers for performance, support, cost and schedule are created

Questions for Assessing overall project risk

1. Have top software and customer managers formally committed to support the project?
2. Are end-users enthusiastically committed to the project?
3. Are requirements fully understood by developers and customers?
4. Were customers fully involved in requirements definition?
5. Do end-users have realistic expectations?
6. Is project scope stable?
7. Does software team have the right skill set?
8. Are project requirements stable?
9. Does the project team have experience with technology to be implemented?
10. Is the number of people on project team adequate to do the job?
11. Do all stakeholders agree on the importance of the project and the requirements for the systems being built?

If any one of the answers to these questions is negative, mitigation, monitoring, and management steps should be instituted

Risk components and drivers

The US Air Force has given guidelines for software risk identification and abatement.

The risk components to be identified are performance, cost, support and schedule

Risk components

- Performance risks -the degree of uncertainty that the product will meet its requirements and be fit for its intended use
- Cost risks- the degree of uncertainty that the project budget will be maintained
- Support risks- the degree of uncertainty that the resultant software will be easy to correct, adapt and enhance
- Schedule risks - the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time

Risk impact

- The impact of each risk component is divided into four categories- negligible, marginal, critical, catastrophic

- The risk drivers affecting each risk component are classified according to their impact category and the potential consequences of each undetected software fault or unachieved project outcome are described

Components Category	Performance	Support	Cost	Schedule
Catastrophic				
Critical				
Marginal				
Negligible				

Risk Projection

- Is also called as Risk Estimation, which attempts to rate each risk in two ways
 - 1) the likelihood or probability that the risk is real and
 - 2) the consequences of the problems associated with the risk, should it occur.
- The four risk projection steps are
 1. Establish a scale that reflects the perceived likelihood of each risk
 2. Delineate the consequences of the risk
 3. Estimate the impact of the risk on the project and product
 4. Note the overall accuracy of the risk projection to avoid misunderstandings

The intent of these steps is to prioritization of risks to allocate resources where they will have the most impact.

Developing a Risk Table

- List all risks in the first column of the table
- Classify each risk and enter the category label in column two (e.g. PS -project size, BU – business risk)
- Determine a probability for each risk and enter it into column three
- Enter the impact (severity) of each risk (1-catastrophic, 2-critical ,3-marginal, 4-negligible,) in column four
- Sort the table by probability and impact value
- Determine the criteria for deciding where the sorted table will be divided into the first priority concerns and the second priority concerns by drawing a cutoff line
- First priority risks must be managed . The column labeled RMMM contains a pointer into a Risk Mitigation, Monitoring, and Management Plan
- Risk probability can be determined by estimation on individual basis and arriving at a consensus value.
- Risk drivers can be assessed on a qualitative probability scale with values like – impossible , improbable, probable and frequent which may also have Mathematical probability values associated with them (e.g 0.75 to 0.95 = probable) .

Risks	Category	Probability	Impact	RMMM
Size estimates may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	

Assessing Risk Impact

- Factors affecting risk consequences - nature (types of problems arising), scope (combines severity with extent of project affected), timing (when and how long impact is felt)
- If costs are associated with each risk table entry a risk exposure metric can be computed ($RE = Probability * Cost$) and added to the risk table.

Risk Assessment

- Define referent levels for each project risk that can cause project termination (performance degradation, cost overrun, support difficulty, schedule slippage).
- Attempt to develop a relationship between each risk triple (risk, probability, impact) and each of the reference levels.
- Predict the set of referent points that define a region of termination, bounded by a curve or areas of uncertainty.
- Try to predict how combinations of risks will affect a referent level.

Risk Refinement

- Process of restating the risks as a set of more detailed risks that will be easier to mitigate, monitor, and manage.
- CTC (condition-transition-consequence) format may be a good representation for the detailed risks (e.g., given that <condition> then there is a concern that (possibly) <consequence>).

Risk Mitigation, Monitoring, and Management

- Risk mitigation (proactive planning for risk avoidance)
- Risk monitoring (assessing whether predicted risks occur or not, ensuring risk aversion steps are being properly applied, collect information for future risk analysis, attempt to determine which risks caused which problems)
- Risk management and contingency planning (actions to be taken in the event that mitigation steps have failed and the risk has become a live problem)

Safety Risks and Hazards

- Risks are also associated with software failures that occur in the field after the development project has ended.
- Computers control many mission critical applications in modern times (weapons systems, flight control, industrial processes, etc.).
- Software safety and hazard analysis are quality assurance activities that are of particular concern for these types of applications

Risk Information Sheets

1. Alternative to RMMM in which each risk is documented individually.
2. Often risk information sheets (RIS) are maintained using a database system.
3. RIS components - risk id, date, probability, impact, description, refinement, mitigation/monitoring, management/contingency/trigger, status, originator, assigned staff member.

•

Assessing Risk Impact

- Factors affecting risk consequences -
 - nature - types of problems arising. e.g. A poorly defined external interface to customer hardware will preclude early design and testing and will likely lead to system integration problems late in a project
 - scope - combines severity with extent of project affected
 - timing -when and how long impact is felt

The following steps are recommended to determine the overall consequences of a risk:

1. Determine the average probability of occurrence value for each risk component
2. Determine the impact for each component using the risk component and impact table
3. Complete the risk table and analyze the results

The overall risk exposure, RE , is determined using the relationship

RE = P X C, where P is the probability of occurrence for a risk, and C is the cost to the project should the risk

e.g. assume that the software team defines a project risk in the following manner

Risk identification : Only 70 percent of the software components scheduled for reuse will infact be integrated into the application. The remaining functionality will have to be custom developed.

Risk Probability : 80 percent

Risk impact: 60 reusable software components were planned. If only 70 percent can be used can be used, 18 components would have to be developed from scratch.

Suppose

Average component is 100 LOC

Cost for each LOC is \$14.00

The overall cost impact to develop components = $18 \times 100 \times 14 = \$25,200$

Risk exposure RE = $0.80 \times 25,200 = \$20,200$

- Risk exposure can be computed for each risk in the risk table, once an estimate of the cost of the risk is made.
- The total risk exposure for all risks above the cutoff line can give the final cost estimate for a project.
- The projection and analysis techniques are applied iteratively as the project proceeds
- The project team revisits the risk table at regular intervals, reevaluating each risk as new circumstances cause its probability and impact to change.
- As a result some new risks may be added , some existing risks may be removed , and some may change their position in the table.

Risk Refinement

- As time passes, more is learnt about the project and the risk, it may be possible to refine the risk into a set of more detailed risks.
- CTC (condition-transition-consequence) format may be a good representation for the detailed risks (e.g., given that <condition> then there is a concern that (possibly) <consequence>).
- Using the CTC format for reuse example, we can write

This general condition can be refined in the following manner

Sub Condition 1: Certain reusable components were developed by third party with no knowledge of internal design standards.

Sub Condition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components

Sub Condition 3: Certain reusable components have been implemented in a language that is not supported on the target environment

The consequences associated with the refined sub conditions remain the same [30 percent must be custom engineered], but the refinement helps to isolate the underlying risks and might lead to easier analysis and response.

Risk Mitigation, Monitoring, and Management

- An effective risk strategy must consider three issues
 - Risk avoidance, Risk monitoring, Risk management and contingency planning
- Risk mitigation (proactive planning for risk avoidance)
- Risk monitoring (assessing whether predicted risks occur or not, ensuring risk aversion steps are being properly applied, collect information for future risk analysis, attempt to determine which risks caused which problems)
- Risk management and contingency planning (actions to be taken in the event that mitigation steps have failed and the risk has become a live problem)

For example,

- assume that high staff turnover is noted as a project risk, r1.
- Based on the past projects, the likelihood, l1, of high turnover is estimated to 0.70 , and impact x1 is projected as critical.
- To mitigate this risk, project management must develop a strategy for reducing turnover. Possible steps could be
 - meet with current staff to determine causes for turnover
 - mitigate those causes that are under our control before the project starts
 - Once the project starts, assume turnover will occur and develop techniques to ensure continuity when people leave
 - Define documentation standards and establish mechanisms to ensure that documents are developed in a timely manner
 - Conduct peer reviews of all work
 - Assign a backup staff member for every critical technologist
- As part of risk monitoring activities, in the case of high staff turnover, the following factors can be monitored
 - General attitude of team members based on project pressures
 - Interpersonal relationships among team members
 - Potential problems with compensation and benefits
 - The availability of jobs within the company and outside it
- Risk management and contingency planning assumes that mitigation efforts have failed and the risk has become a reality. A number of people announce that they will be leaving.
- If the mitigation strategy has been followed, backup is available, information is documented, and knowledge has been dispersed across the team.
- Those individuals who are leaving are asked to stop all work and spend their last weeks in knowledge transfer mode.
- Experience indicates that 80 percent of the overall project risk can be accounted for by only 20 percent of the identified risks.
- The work performed during the earlier risk analysis steps will help the planner to determine which of the risks reside in that 20 percent.

Safety Risks and Hazards

- Risks are also associated with software failures that occur in the field after the development project has ended.
- Computers control many mission critical applications in modern times (weapons systems, flight control, industrial processes, etc.).
- Software safety and hazard analysis are quality assurance activities that are of particular concern for these types of applications

The RMMM plan

- A risk management strategy can be included in the software project plan or the risk management steps can be organized into a separate RMMM plan.
- Some teams do not develop a formal RMMM document. Instead, each risk is documented individually using a risk information sheet(RIS).
- In most cases RIS is maintained using a database system.
- Once the RMMM has been documented and the project has begun, risk mitigation and monitoring steps start.
- The three primary objectives of risk monitoring are
 - 1)to assess whether predicted risks occur
 - 2)to ensure that risk aversion steps are properly being applied
 - 3)to collect information that can be used for future risk analysis

Risk Information Sheets

1. Alternative to RMMM in which each risk is documented individually.
2. Often risk information sheets (RIS) are maintained using a database system.
3. RIS components - risk id, date, probability, impact, description, refinement, mitigation/monitoring, management/contingency/trigger, status, originator, assigned staff member.

Software quality assurance

Overview :

This chapter provides an introduction for software quality management. Software quality assurance (SQA) is the concern of every software engineer to reduce cost and improve product time-to-market. A Software Quality Assurance Plan is not merely another name for a test plan, though test plans are included in an SQA plan. SQA activities are performed on every software project. Use of metrics is an important part of developing a strategy to improve the quality of both software processes and work products.

Quality Concepts

- Variation control is the heart of quality control (software engineers strive to control the process applied, resources expended, and end product quality attributes).
- Quality of design - refers to characteristics designers specify for the end product to be constructed
- Quality of conformance - degree to which design specifications are followed in manufacturing the product
- Quality control - series of inspections, reviews, and tests used to ensure conformance of a work product to its specifications
- Quality assurance - consists of the auditing and reporting procedures used to provide management with data needed to make proactive decisions

Cost of Quality

- Prevention costs - quality planning, formal technical reviews, test equipment, training
- Appraisal costs - in-process and inter-process inspection, equipment calibration and maintenance, testing
- Failure costs - rework, repair, failure mode analysis
- External failure costs - complaint resolution, product return and replacement, help line support, warranty work

Software Quality Assurance

1. Conformance to software requirements is the foundation from which software quality is measured.
2. Specified standards are used to define the development criteria that are used to guide the manner in which software is engineered.
3. Software must conform to implicit requirements (ease of use, maintainability, reliability, etc.) as well as its explicit requirements.

SQA Group Activities

- Prepare SQA plan for the project.
- Participate in the development of the project's software process description.
- Review software engineering activities to verify compliance with the defined software process.
- Audit designated software work products to verify compliance with those defined as part of the software process.
- Ensure that any deviations in software or work products are documented and handled according

- to a documented procedure.
- Record any evidence of noncompliance and reports them to management.

Software Reviews

- Purpose is to find errors before they are passed on to another software engineering activity or released to the customer.
- Software engineers (and others) conduct formal technical reviews (FTRs) for software engineers.
- Using formal technical reviews (walkthroughs or inspections) is an effective means for improving software quality.

Software Defects

- Industry studies suggest that design activities introduce 50-65% of all defects or errors during the software process
- Review techniques have been shown to be up to 75% effective in uncovering design flaws which ultimately reduces the cost of subsequent activities in the software process
- Defect amplification models can be used to show that the benefits of detecting and removing defects from activities that occur early in the software process

Formal Technical Reviews

- Involves 3 to 5 people (including reviewers)
- Advance preparation (no more than 2 hours per person) required
- Duration of review meeting should be less than 2 hours
- Focus of review is on a discrete work product
- Review leader organizes the review meeting at the producer's request
- Reviewers ask questions that enable the producer to discover his or her own error (the product is under review not the producer)
- Producer of the work product walks the reviewers through the product (alternative: in inspections a "reader" who is not the producer presents the work product)
- Recorder writes down any significant issues raised during the review
- Reviewers decide to accept or reject the work product and whether to require additional reviews of product or not

Formal Technical Review Guidelines

1. Review the product not the producer.
2. Set an agenda and maintain it.
3. Limit rebuttal and debate.
4. Enunciate problem area, but don't attempt to solve every problem noted.
5. Take written notes.
6. Limit number of participants and insist on advance preparation.
7. Develop a checklist for each product that is likely to be reviewed.
8. Allocate resources and schedule time for all reviewers.
9. Conduct meaningful training for all reviewers.
10. Review your early reviews,

Sample Driven Reviews

- Samples of all software engineering work products are reviewed to determine the most error-prone
- Full FTR resources are focused on the likely error-prone work products based on sampling results
- To be effective the sample driven review process must be driven by quantitative measures of the work products
 - Inspect a representative fraction of the content of each software work product (i) and record the number of faults (fi) found within (ai)
 - Develop a gross estimate of the number of faults within work product i by multiplying fi by 1/ai
 - Sort work products in descending order according to the gross estimate of the number of faults in each
 - Focus on available review resources on those work products with the highest estimated number of faults

Statistical Quality Assurance

1. Information about software defects is collected and categorized
2. Each defect is traced back to its cause
3. Using the Pareto principle (80% of the defects can be traced to 20% of the causes) isolate the "vital few" defect causes
4. Move to correct the problems that caused the defects in the "vital few"

Six Sigma Software Engineering

- Define customer requirements, deliverables, and project goals via well-defined methods of customer communication.
- Measure each existing process and its output to determine current quality performance (e.g., compute defect metrics)
- Analyze defect metrics and determine viral few causes.
- For an existing process that needs improvement
 - Improve process by eliminating the root causes for defects
 - Control future work to ensure that future work does not reintroduce causes of defects
- If new processes are being developed
 - Design each new process to avoid root causes of defects and to meet customer requirements
 - Verify that the process model will avoid defects and meet customer requirements

Software Reliability

- Defined as the probability of failure free operation of a computer program in a specified environment for a specified time period
- Can be measured directly and estimated using historical and developmental data (unlike many other software quality factors)
- Software reliability problems can usually be traced back to errors in design or implementation.
- Measures of Reliability
 - Mean time between failure (MTBF) = MTTF + MTTR

- ❑ MTTF = mean time to failure
- ❑ MTTR = mean time to repair
- ❑ Availability = $[\text{MTTF} / (\text{MTTF} + \text{MTTR})] \times 100\%$

Software Safety

- Defined as a software quality assurance activity that focuses on identifying potential hazards that may cause a software system to fail.
- Early identification of software hazards allows developers to specify design features can eliminate or at least control the impact of potential hazards.
- Software reliability involves determining the likelihood that a failure will occur, while software safety examines the ways in which failures may result in conditions that can lead to a mishap.

Mistake-Proofing Software

- Poka-yoke devices are mechanisms that lead to the prevention of a potential quality problem before it occurs or to the rapid detection of a quality problem if one is introduced
- Poka-yoke devices are simple, cheap, part of the engineering process, and are located near the process task where the mistakes occur

ISO 9000 Quality Standards

- Quality assurance systems are defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management.
- ISO 9000 describes the quality elements that must be present for a quality assurance system to be compliant with the standard, but it does not describe how an organization should implement these elements.
- ISO 9001:2000 is the quality standard that contains 20 requirements that must be present in an effective software quality assurance system.

SQA Plan

- Management section - describes the place of SQA in the structure of the organization
- Documentation section - describes each work product produced as part of the software process
- Standards, practices, and conventions section - lists all applicable standards/practices applied during the software process and any metrics to be collected as part of the software engineering work
- Reviews and audits section - provides an overview of the approach used in the reviews and audits to be conducted during the project
- Test section - references the test plan and procedure document and defines test record keeping requirements
- Problem reporting and corrective action section - defines procedures for reporting, tracking, and resolving errors or defects, identifies organizational responsibilities for these activities
- Other - tools, SQA methods, change control, record keeping, training, and risk management